Recursion Algorithm

Subject: Introduction to Algorithm

Lecturer: Hoang Do Thanh Tung

Credit points	3 ECTS
Level	Undergraduate
Teaching time	2021
Location	University of Science and Technology of Hanoi

- Simplify and delegate
- Tower of Hanoi puzzle
- Recursion Algorithm
- Exercises
- Practice

Something be important first



Algorithm design process

- **Problem Analysis:** Comprehending the problem's requirements and constraints
- **Algorithm Design:** The chosen strategy is used to construct the algorithm's structure and logic.
- **Complexity Analysis:** Evaluate the algorithm's efficiency based on the time and memory required for execution.

Algorithm Strategies

Strategy	Purpose	Suitable Problems	Efficiency	Advantages	Disadvantages	Other Characteristics	Common Characteristics
Strategy: Backtracking							
	Find all solutions to a problem	Problems that can be represented as a decision tree	Finds all solutions	Easy to implement	Can be time-consuming and memory-intensive for problems with many solutions		
Strategy: Branch and Bound							
	Find the optimal solution for problems with many choices	Problems that can be represented as a decision tree	Optimal	Efficient for problems with many choices	Can be time-consuming and memory-intensive for problems with many branches	Uses the idea of pruning unpromising branches	
Strategy: Divide and Conquer (DC)							
	Solve a problem by dividing it into smaller parts, solving each part, and then combining the results of the smaller parts to solve the original problem	Problems with a tree structure or that can be divided into independent parts	Logarithmic (log n)	Easy to implement, efficient for large problems	Can be time-consuming and memory-intensive for problems with a complex structure	Uses the idea of divide and conquer	
Strategy: Dynamic Programming (DP)							
	Find the optimal solution for problems with a recursive structure and overlapping subproblems	Problems that can be broken down into subproblems with a similar structure	Optimal	Easy to implement for problems with a clear structure	Can be time-consuming and memory-intensive for large problems	Stores the results of previously computed subproblems	Uses the idea of divide and conquer
Strategy: Greedy							
	step	Problems that can be broken down into independent decisions	Does not guarantee an optimal solution, but often efficient for large problems	Easy to implement, efficient for large problems	Does not guarantee an optimal solution	Chooses the best option at each step	

Bộ lọc nhóm theo" của bạn không đồng bộ Làm mới ×

They are foundation for Algorithms

- **DP:** Utilized in optimization algorithms like Reinforcement Learning, Hidden Markov Models.
- **DC:** Employed in natural language processing, image recognition algorithms.
- **Greedy:** Used in data clustering, feature selection algorithms.
- **Branch and Bound:** Applied in hyperparameter optimization, finding optimal solutions for NP-hard problems.
- **Backtracking:** Used in state space search, solving Sudoku puzzles

Prerequisite

Recursion algorithms

Suggestion for students for this section

- You are able to program with any language, such as Pascal, C, Java, Python.... and you did learn "function call" method
- You will study with your computer in which a familiar programming software is installed

Function call?

- a "function call" refers to the process of invoking or executing a function
  - use a defined function and pass any necessary arguments
  - transfers control to the function's code, and then returns control back to the point where the function was called.
- The general syntax for a function call depends on the programming language

```python
# Python Built-in function call
print("Hello, world!")

# User-defined function call
def add(a, b):
    return a + b

result = add(5, 3)
print(result)
```

```javascript
// JavaScript Built-in function call
console.log("Hello, world!");

// User-defined function call
function add(a, b) {
    return a + b;
}

var result = add(5, 3);
console.log(result);
```

```cpp
#include <iostream>

// Function declaration
void greet(const char* name);

int main() {
    // C++ Built-in function call
    std::cout << "Hello, world!" << std::endl;

    // User-defined function call
    greet("Alice");

    return 0;
}

// Function definition
void greet(const char* name) {
    std::cout << "Hello, " << name << "!" << std::endl;
}
```

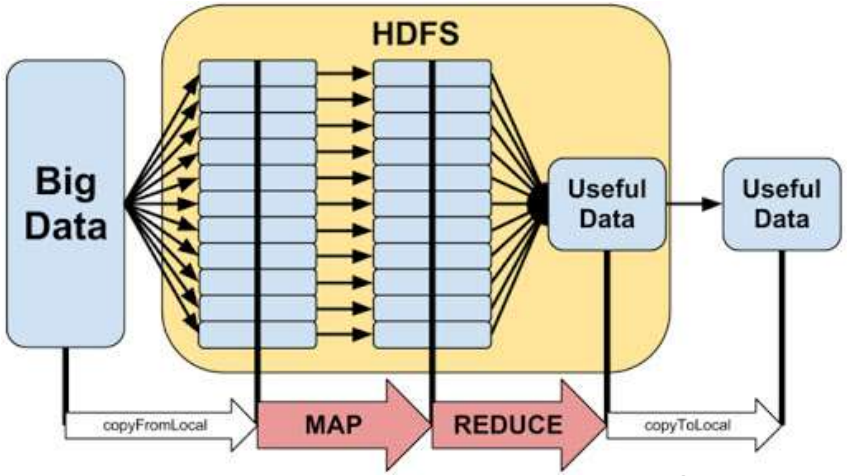
Simplify and delegate



Reduction to simplify a problem

- *Reduction* is the single most common technique used in designing algorithms
- Reducing one problem *X* to another problem Y (or set of problems) means
  - to write an algorithm for *X*, using an algorithm for *Y* as a *subroutine or black box*.





Simplify and delegate

- Simplify complex tasks by breaking them down into smaller, more manageable parts
- and delegate those parts to separate functions or components
- *Ex: to calculate the total price of a shopping cart, including taxes and discounts*

```
func calculate_subtotal(items)
    subtotal = 0
    for item in items:
        subtotal += item['price'] * item['quantity']
    return subtotal

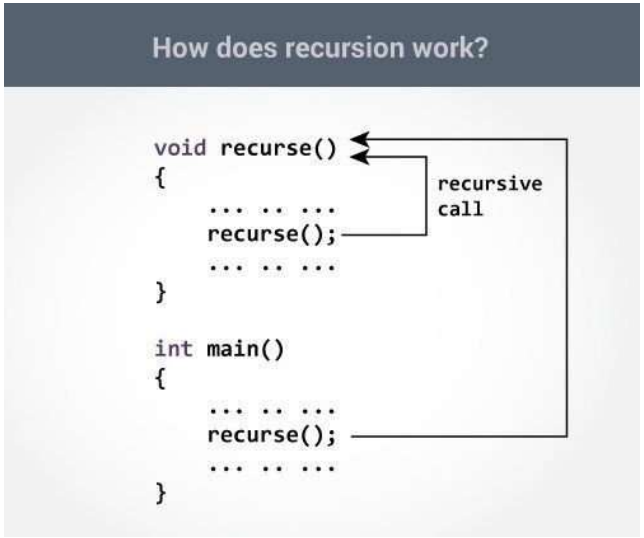
func calculate_tax(subtotal, tax_rate)
    tax = subtotal * tax_rate
    return tax

func apply_discount(subtotal, discount)
    discount_amount = subtotal * discount
    return discount_amount

func calculate_total(items, tax_rate, discount)
    subtotal = calculate_subtotal(items)
    tax = calculate_tax(subtotal, tax_rate)
    discount_amount = apply_discount(subtotal, discount)
    total = subtotal + tax - discount_amount
    return total
```

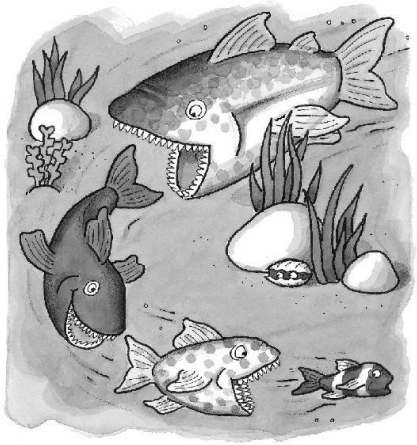
Recursion to simplify and delegate

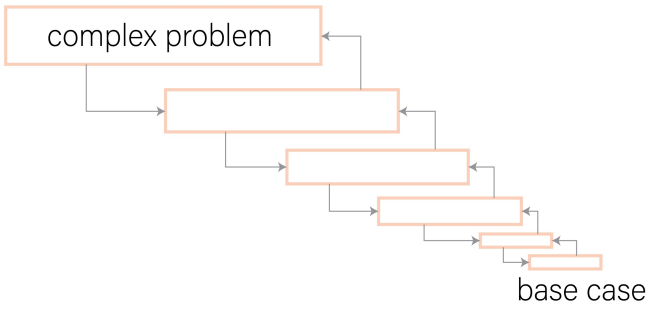
- *Recursion is a* particularly powerful kind of reduction, where a function calls itself (recursive calls) to solve smaller instances of the same problem
- *Recursion* can be defined loosely as follows:
  - **Base Case**: If the given problem is small or simple enough, just solve it. Otherwise,
  - **Simplification:** Reduce the problem to one or more *simpler* ***instances of the same problem and*** 'simpler' and 'simpler' subproblems
  - **Delegation**: Assigning these subproblems to recursive calls
- **Correctness** of a Recursive algorithm is that it must stop and never recurses forever



If you are confused how it works, ..

- **Method of thinking:**
  - **Delegate:** Imagine that someone else (the "Recursion Fairy") is going to solve the simpler problems
  - **Simplify:** Your only task is to simplify/break down the original problem into one or more simpler subproblems
  - **Base Case:** Define a base case to be solved directly without any further recursion





Tower of Hanoi puzzle



Remarkable story

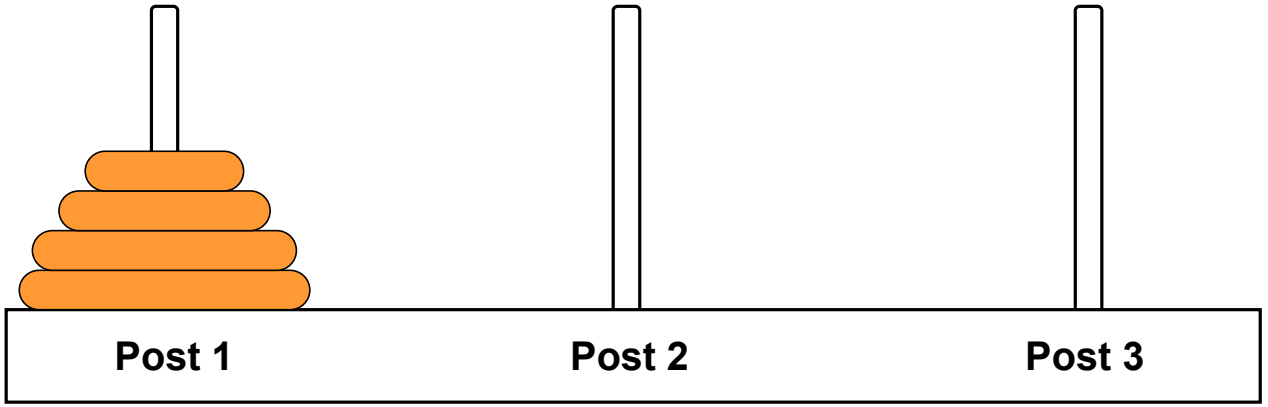
- The Tower of Hanoi puzzle was first published by the mathematician François Éduoard Anatole Lucas in 1883,
  - In the great temple, there were fixed three diamond needles.



  - On one of these needles, at the creation,
    - **God** placed 64 discs of pure gold,
    - the discs is smaller and smaller up to the top one.
  - The priests transfer the discs from one needle to another according to laws,
    - must not move more than one disc at a time, and
    - must place this disc on a needle so that there is no smaller disc below it.
  - When the 64 discs are moved to another needle is time the world will vanish

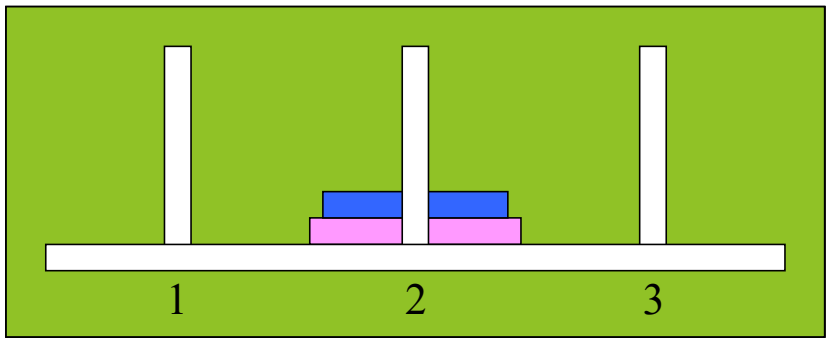


Towers of Hanoi

- Task: Move n disks from current post to another post.
- 

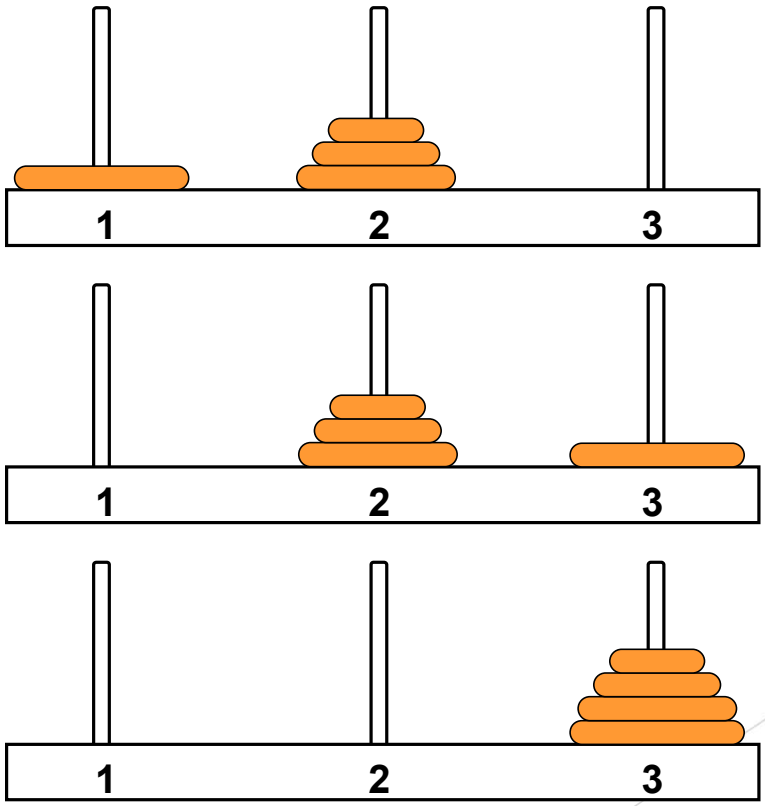
- Rules:
- (1) Can only move one disk at a time.
- (2) A larger disk can never be placed on top of a smaller disk.
  (3) May use third post for temporary storage.

Solution for n = 2



How is about n>2 ?

- Suppose disks start on Post 1, and target is Post 3.
- 1. Move top n-1 disks to Post 2.
- 2. Move largest disk to Post 3.
- 3. Move n-1 disks from Post 2 to Post 3.

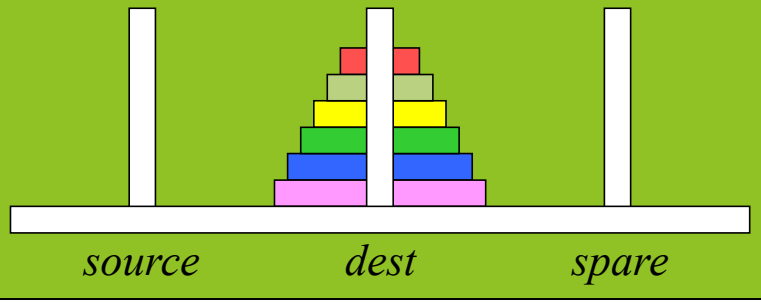


Task Decomposition (cont.)

- Task 1 is really the same problem,
  with fewer disks and a different target post.
  - "Move n-1 disks from Post 1 to Post 2."
- And Task 3 is also the same problem,
  with fewer disks and different starting and target posts.
  - "Move n-1 disks from Post 2 to Post 3."
- So this is a recursive algorithm.
  - The terminal case is moving the smallest disk -- can move directly without using third post.
  - Number disks from 1 (smallest) to n (largest).

Algorithm for n = 6

1. If $n = 1$:
   1.1. Move a single disk from *source* to *dest*.
2. If $n > 1$:
   2.1. Let *spare* be the remaining pole, other than *source* and *dest*.
   2.2. Move a tower of (n–1) disks from *source* to *spare*.
   2.3. Move a single disk from *source* to *dest*.
   2.4. Move a tower of (n–1) disks from *spare* to *dest*.
3. Terminate.



Towers of Hanoi: Pseudocode

```
MoveDisk(diskNumber, startPost, endPost, midPost)
{
  if (diskNumber > 1) {
    /* Move top n-1 disks to mid post */
    MoveDisk(diskNumber-1, startPost, midPost, endPost);

    printf("Move disk number %d from %d to %d.\n",
           diskNumber, startPost, endPost);

    /* Move n-1 disks from mid post to end post */
    MoveDisk(diskNumber-1, midPost, endPost, startPost);
  }
  else
    Move 1 disk from startPost to endPost;
    printf("Move disk number 1 from %d to %d.\n",
           startPost, endPost);
}
```

Analyse the algorithm

- Analysis (counting moves):

  Let the total no. of moves required to move a tower of n disks be $moves(n)$. Then:

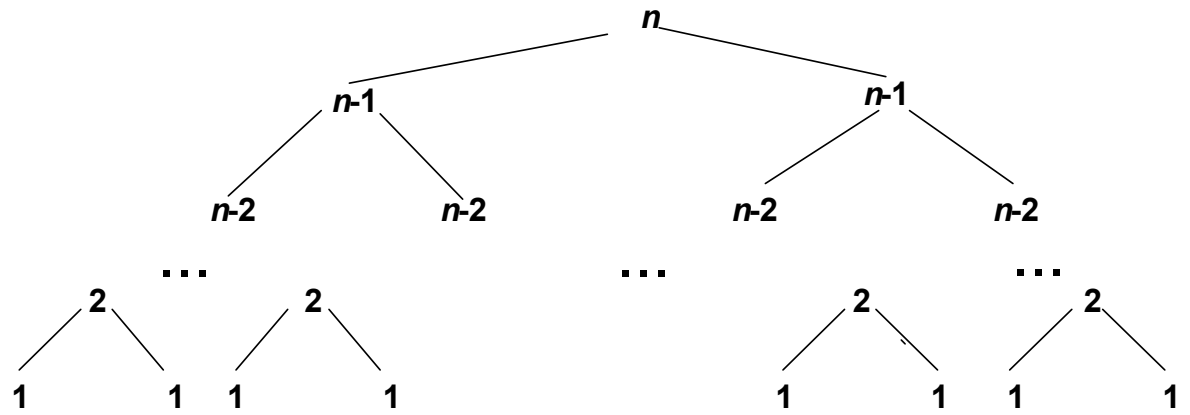
  $moves(n) = 1$ if $n = 1$
  $moves(n) = 1 + 2*moves(n-1)$ if $n > 1$

  Solution:

  $moves(n) = 2^n - 1$

  Time complexity is $O(2^n)$.

Tree of calls for the Tower of Hanoi Puzzle



When will the world vanish?

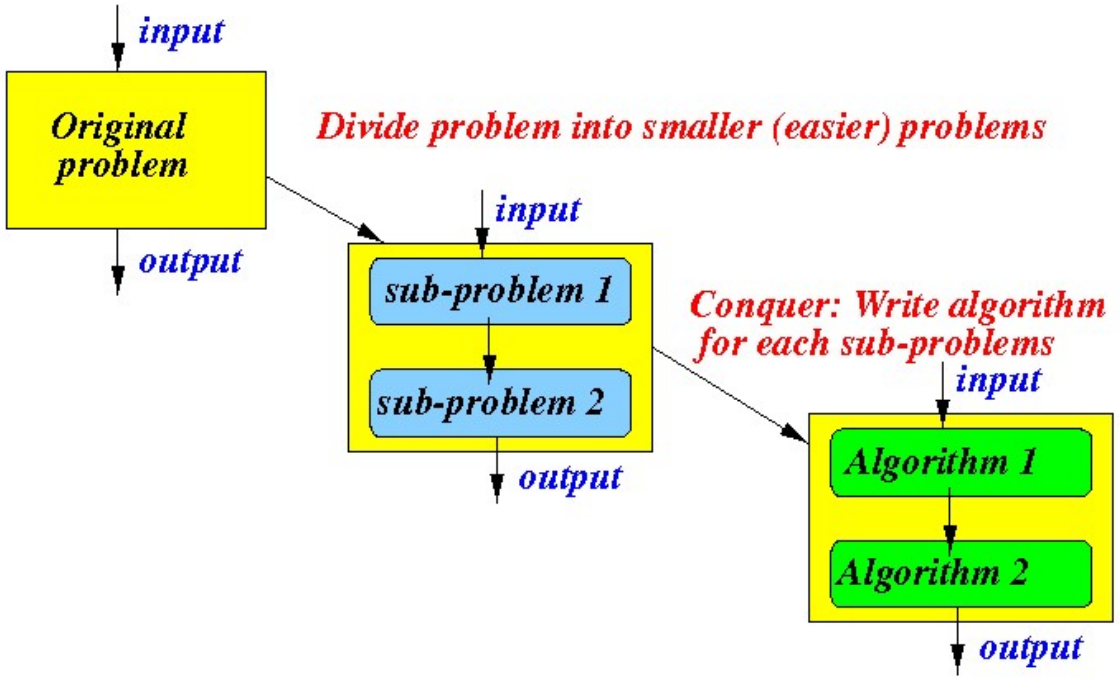


- Moving a tower of 64 disks
  requires $2^{64} - 1 = 18,446,744,073,709,551,615$ individual moves
- Impressive rate of one move per second, the monks will be at work for approximately **585 billion years**

Recursion algorithm



Divide and Conquer



Recursion

- **a special *divide* and *conquer* problem solving technique**

- **Wait for *someone else*** to solve the **problem** of **size *n* − *1***, **size *n* − *2***, etc...

- When you **receive** the **solution** for the **problem** of **size *n* − *1***, **size *n* − *2***, etc..., you will then ***use*** these **solutions** to **solve the *original* problem**.

Recursive problem

- ***Recursive*** **problem** = a **problem** that is defined with **one or more** ***smaller*** **problems** of the ***same*** **kind**

- **Example:** the **factorial** of the number **n**

$$n! = n \times (n-1)!$$

where:

$$0! = 1$$

Example

- **Now**, ***suppose*** that you ***know*** (e.g., someone told you) that:

$$9! = 362880$$

- Then, we can solve the **original problem** (**10!**) ***very*** **easily**:

$$\begin{aligned} 10! &= 10 \times 362880 \\ &= 3628800 \end{aligned}$$

The *Recursive* Property

A **problem *X(n)*** has the **recursive property** if:

- The **problem *X(n)*** can be **solved *using*** the **solution** of ***one or more smaller*** **problems *X(n−1)***, ***X(n−2)***, ...
- A ***sufficient*** **number** of **base (simple) cases** of the **problem** can be **solved *readily***.

Example

The **problem** ***factorial(n)*** can be **solved** ***using*** the **solution** of ***one smaller*** **problem** ***factorial(n−1)***

Because ***one*** **smaller problem** is used to solve **factorial(n)**, we need ***one*** **base case**

The **base (simple) case** ***n = 0*** of the **factorial problem** can be **solved** ***readily***:

factorial(0) = 1

Design Algorithm for a *recursive* problem

1. Find out ***which* smaller problems** you need to use to solve the ***original* problem**

2. Find out ***how*** to **use** the **solutions** of the **smaller problems** to **solve the *original* problem**.

3. Find the **solutions** for a ***sufficient* number** of the **base cases**.

Computing *n!*

- Which ***smaller* problem** do we use to solve **factorial(n)**:

$$sol_1 = factorial(n-1)$$

- ***How*** do we **use** the **solution sol_1** to solve **factorial(n)**:

$$factorial(n) = n \times sol_1$$

- Because we used **factorial(n−1)**, we will need the **solution** for **1 base case**:

$$factorial(0) = 1$$

Computing *n! algorithm*

```
int factorial( int n ) {
        if ( n <= 1 ) {
            return 1;
        } else {
            return n * factorial( n – 1 );
        }
    }
```

```
public class Recurse1
{
  public static int factorial( int n )
  {
    int sol1, mySol;     // Define variable...

    if ( n == 0 /* the base cases */ )
    {
      return 1; // The readily available solution for the base case
    }
    else
    {
      sol1 = factorial ( n-1 );  // Let "someone else" (= factorial(n-1) )
                                 // solve the smaller problem

      mySol = n * sol1;       // Use the solution of the smaller problem
                              // to solve my own problem

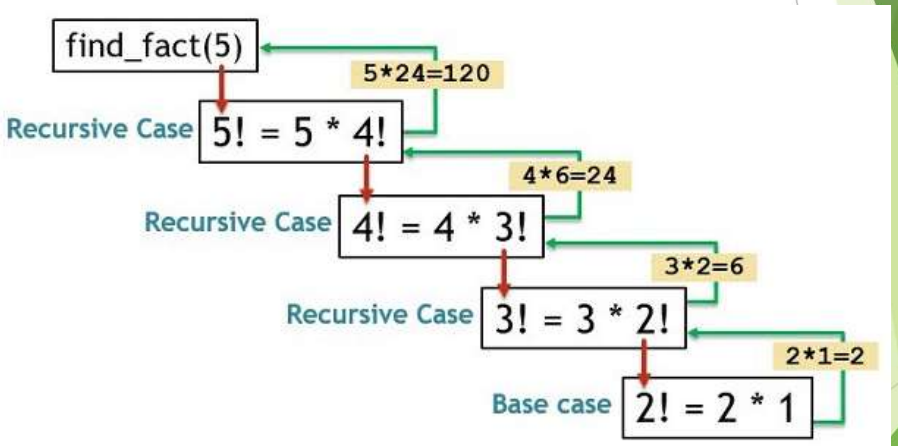
      return mySol;     // Return my solution
    }
  }
}
```

```
        int factorial( int n ) {
        return (n <= 1) ? 1 : n * factorial( n – 1 );
    }
```

How to analyze Recursive algorithm?

- If there are function calls, these must be analyzed first
- If there are recursive functions, be careful about their analyses. For some recursions, the analysis is trivial

```
long factorial(int n){
 if (n <= 1)
     return 1;
  return n * factorial(n-1);
}
```



Analyze the fact algorithm (1)

Thus, we may analyze the run time of this function as follows:

$$T_!(n) = \begin{cases} \Theta(1) & n \leq 1 \\ T_!(n-1) + \Theta(1) & n > 1 \end{cases}$$

The analysis of the run time of this function yields a recurrence relation:

$$T_!(n) = T_!(n-1) + \Theta(1) \qquad T_!(1) = \Theta(1)$$

Replace each Landau symbol with a representative function:

$$T_!(n) = T_!(n-1) + 1 \qquad T_!(1) = 1$$

Analyze the fact algorithm (2)

we can examine the first few steps:

$$T_!(n) = T_!(n-1) + 1$$
$$= T_!(n-2) + 1 + 1 = T_!(n-2) + 2$$
$$= T_!(n-3) + 3$$

From this, we see a pattern:

$$T_!(n) = T_!(n-k) + k$$

Analyze the fact algorithm(3)

If $k = n - 1$ then

$$\begin{aligned} T_!(n) &= T_!(n - (n - 1)) + n - 1 \\ &= T_!(1) + n - 1 \\ &= 1 + n - 1 = n \end{aligned}$$

Thus, $T_!(n) = \Theta(n)$

Merge-Sort

- Merge-sort on an input sequence S with n elements consists of three steps:
  - Divide: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
  - Recur: recursively sort S_1 and S_2
  - Conquer: merge S_1 and S_2 into a unique sorted sequence

Algorithm *mergeSort*(*S*, *C*)
  Input sequence *S* with *n* elements, comparator *C*
  Output sequence *S* sorted according to *C*
  if *S.size*() > 1
    (*S*$_1$, *S*$_2$) ← *partition*(*S*, *n*/2)
    mergeSort(*S*$_1$, *C*)
    mergeSort(*S*$_2$, *C*)
    S ← *merge*(*S*$_1$, *S*$_2$)

Merging Two Sorted Sequences

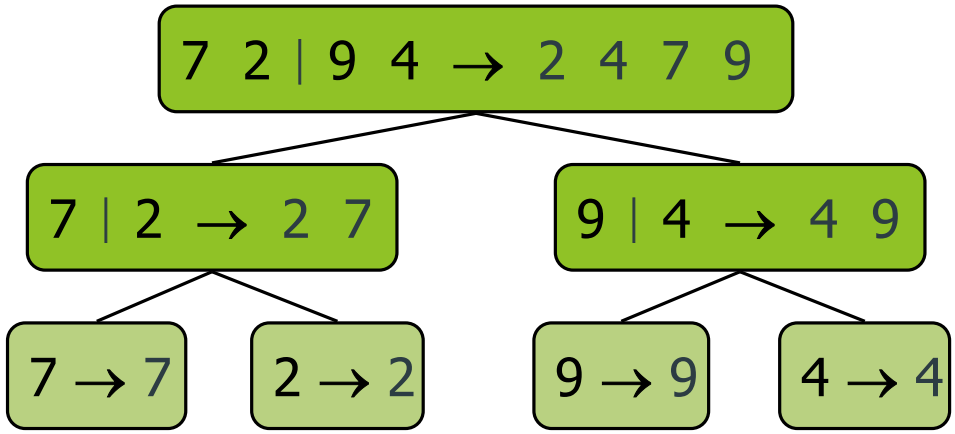
- The conquer step of merge-sort consists of merging two sorted sequences A and B into a sorted sequence S containing the union of the elements of A and B
- Merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes $O(n)$ time

```
Algorithm merge(A, B)
    Input sequences A and B with
        n/2 elements each
    Output sorted sequence of A ∪ B

    S ← empty sequence
    while ¬A.isEmpty() ∧ ¬B.isEmpty()
        if A.first().element() < B.first().element()
            S.insertLast(A.remove(A.first()))
        else
            S.insertLast(B.remove(B.first()))
    while ¬A.isEmpty()
        S.insertLast(A.remove(A.first()))
    while ¬B.isEmpty()
        S.insertLast(B.remove(B.first()))
    return S
```

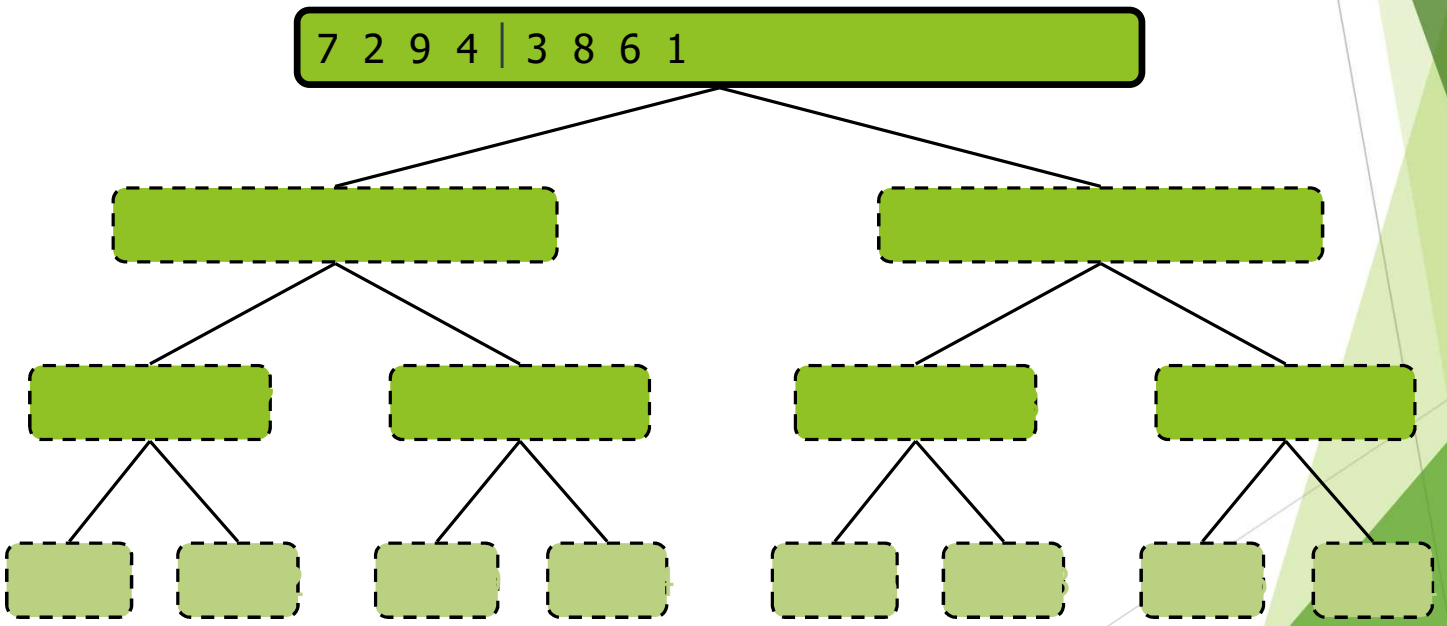
Merge-Sort Tree

- An execution of merge-sort is depicted by a binary tree
  - each node represents a recursive call of merge-sort and stores
    - unsorted sequence before the execution and its partition
    - sorted sequence at the end of the execution
  - the root is the initial call
  - the leaves are calls on subsequences of size 0 or 1



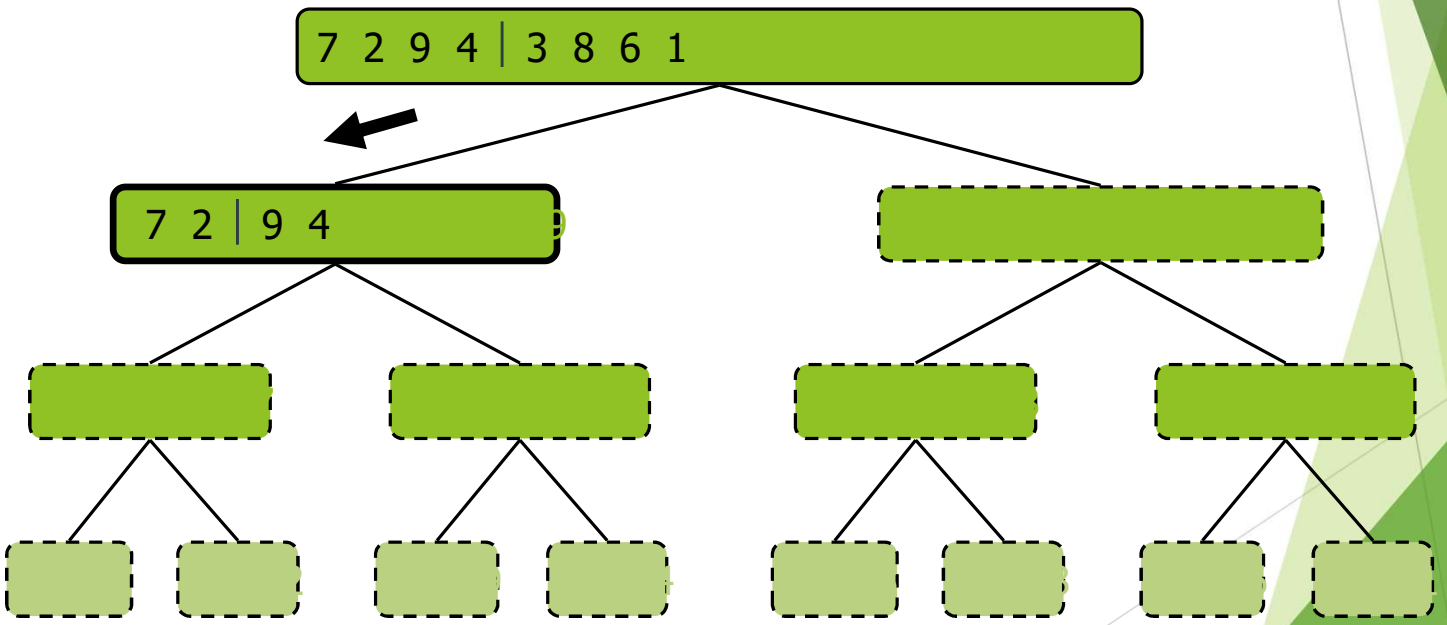
Execution Example

- Partition



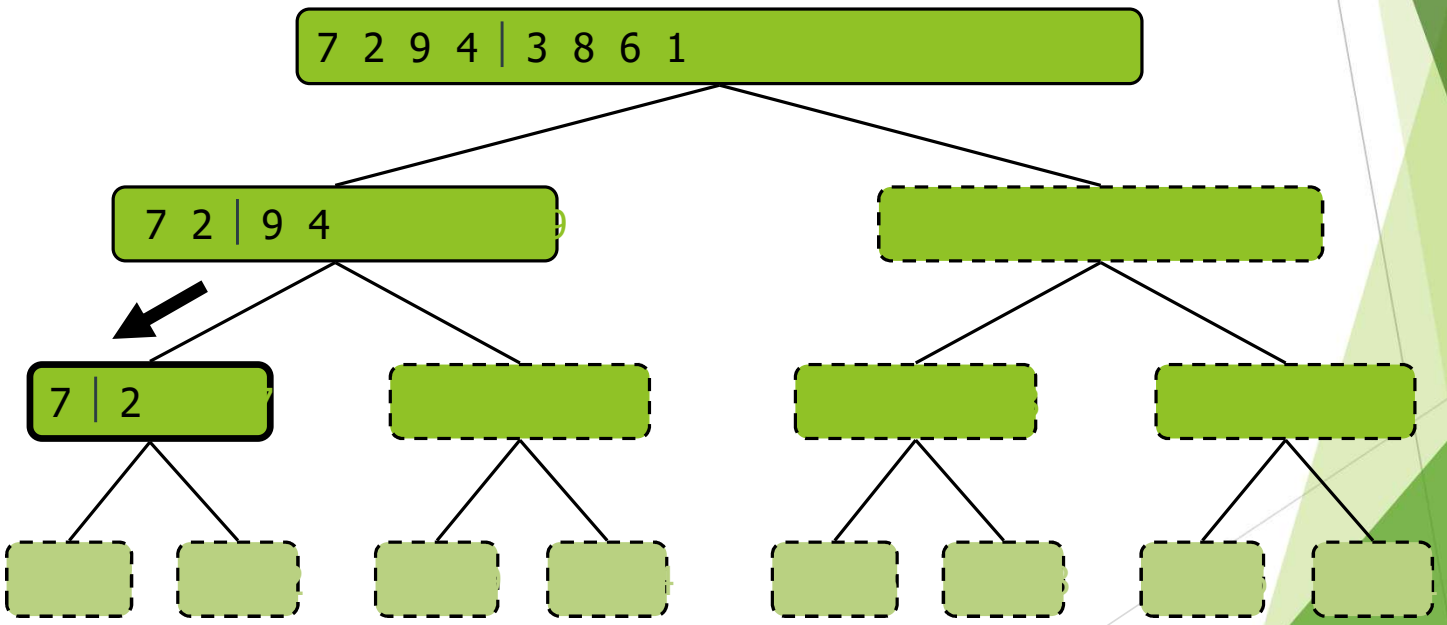
Execution Example (cont.)

- Recursive call, partition



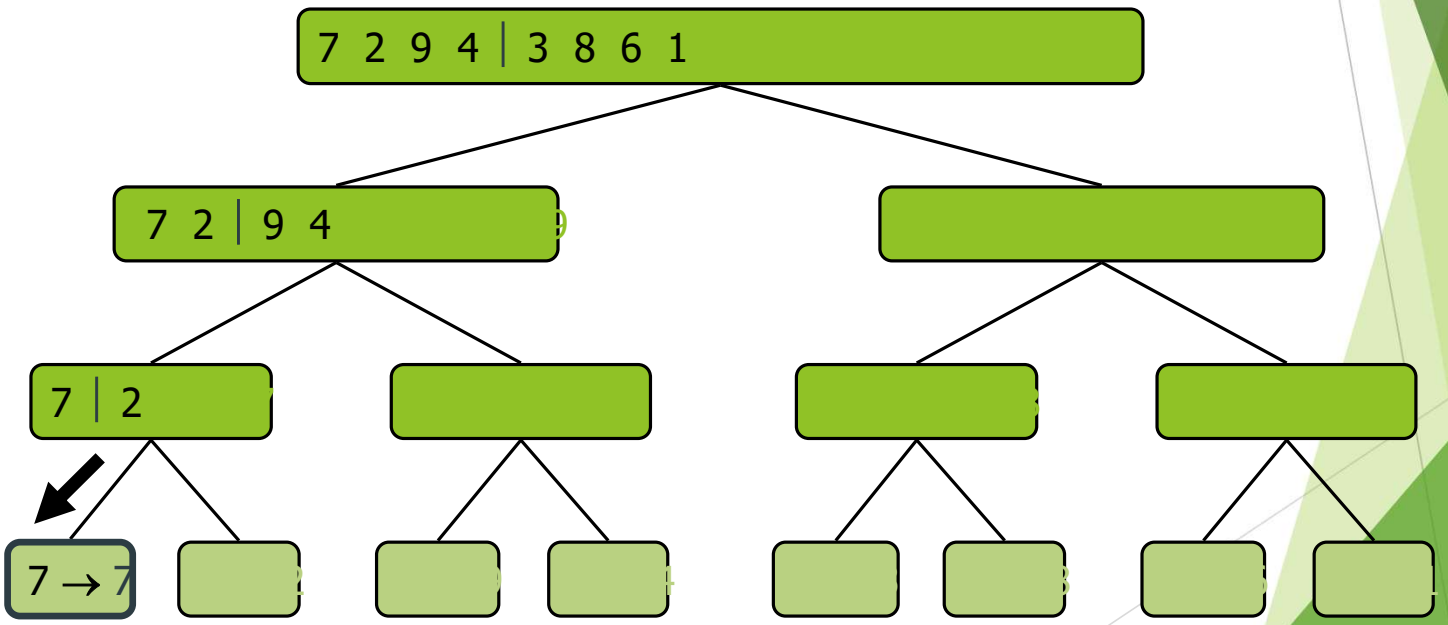
Execution Example (cont.)

- Recursive call, partition



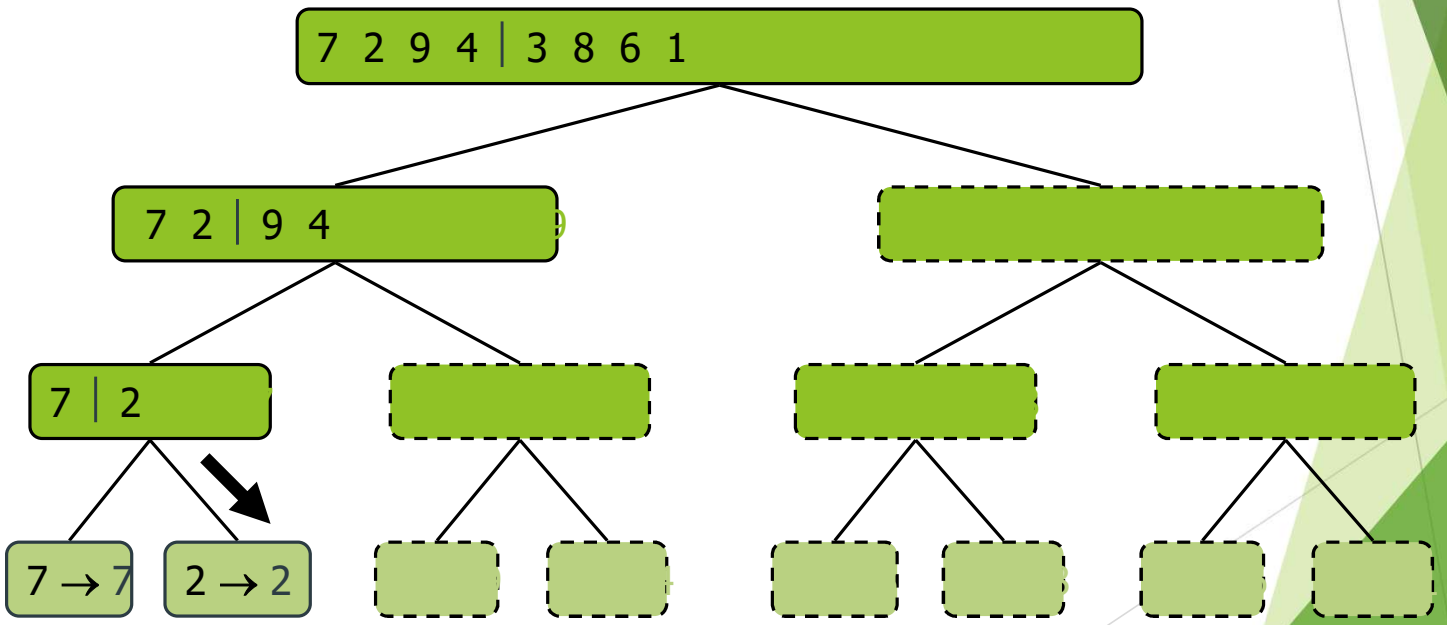
Execution Example (cont.)

- Recursive call, base case



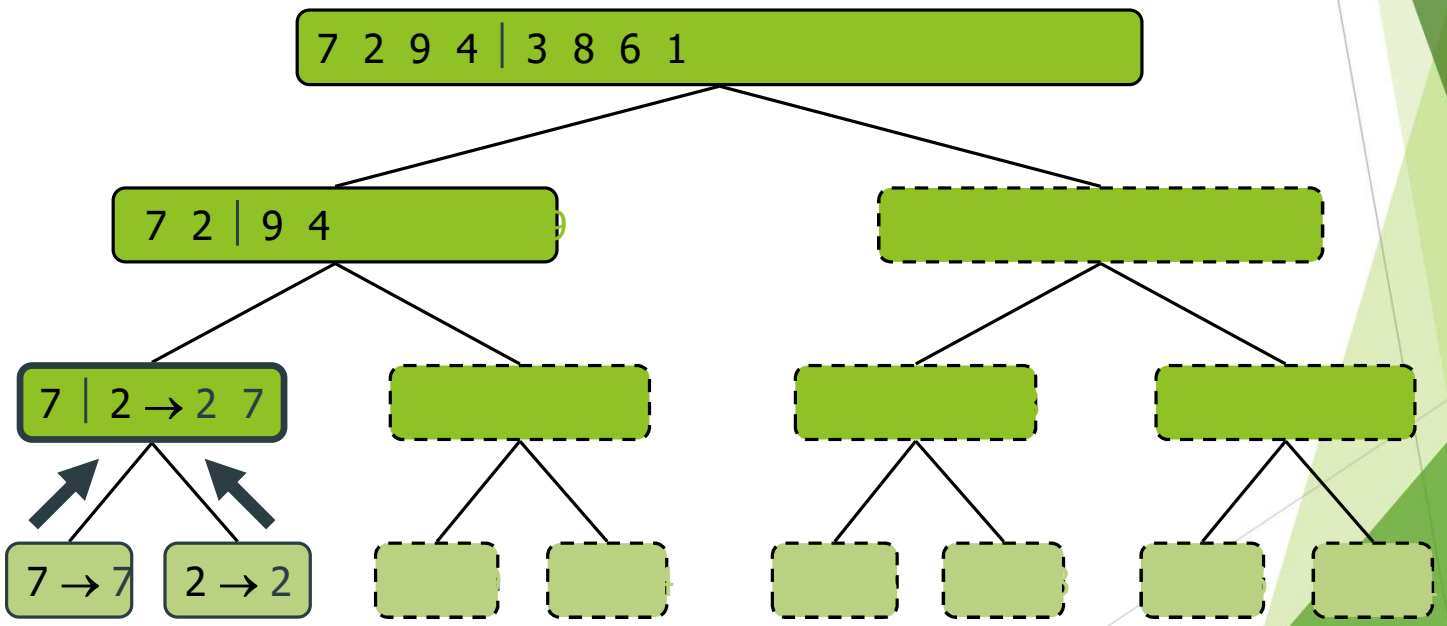
Execution Example (cont.)

- Recursive call, base case



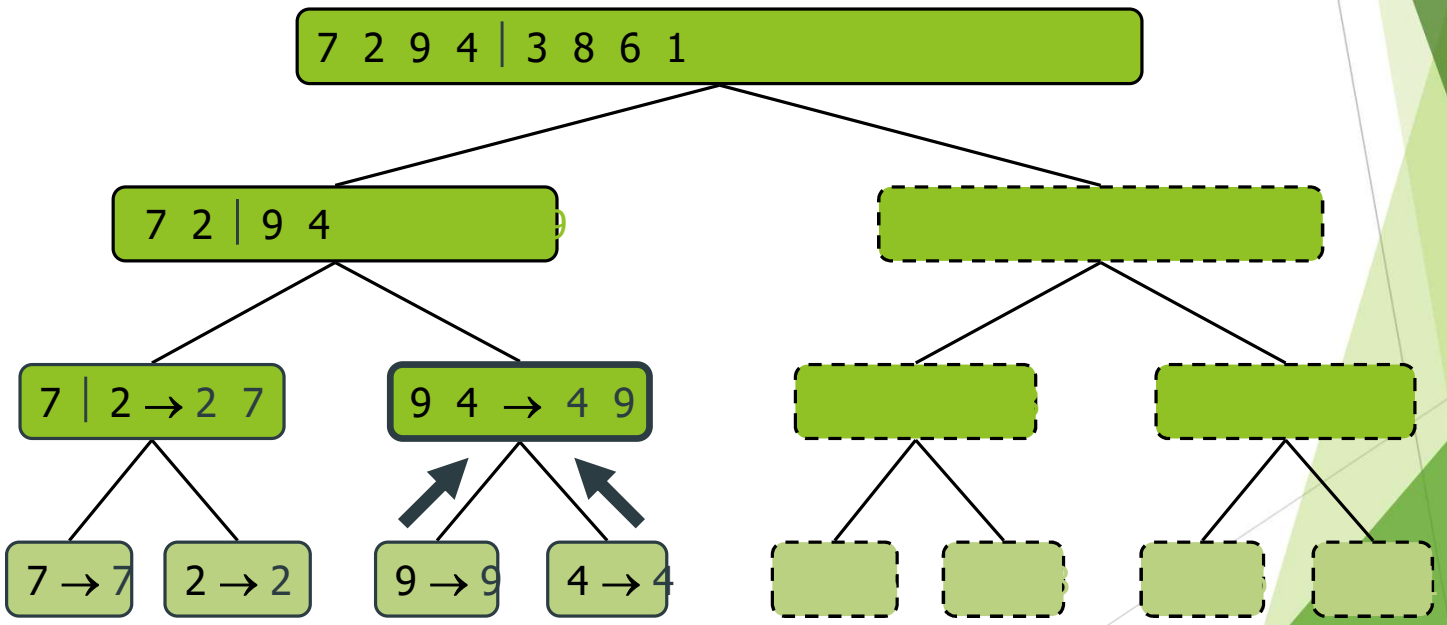
Execution Example (cont.)

- Merge



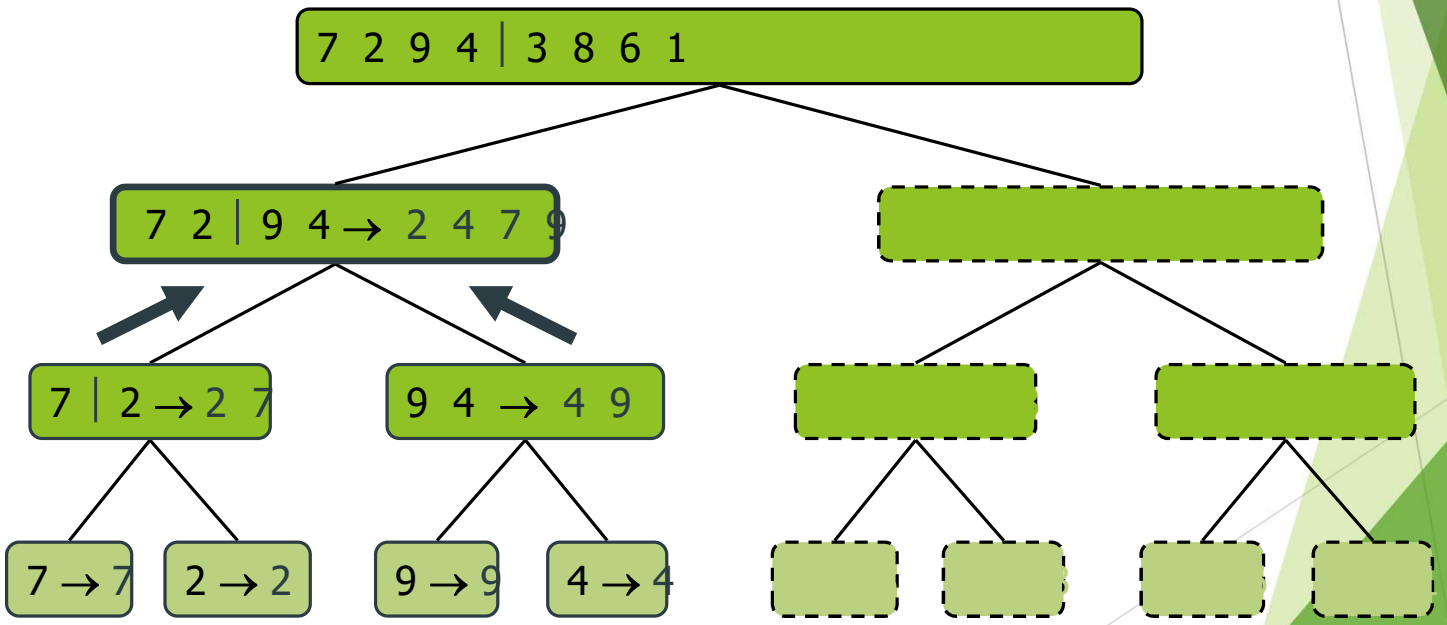
Execution Example (cont.)

- Recursive call, …, base case, merge



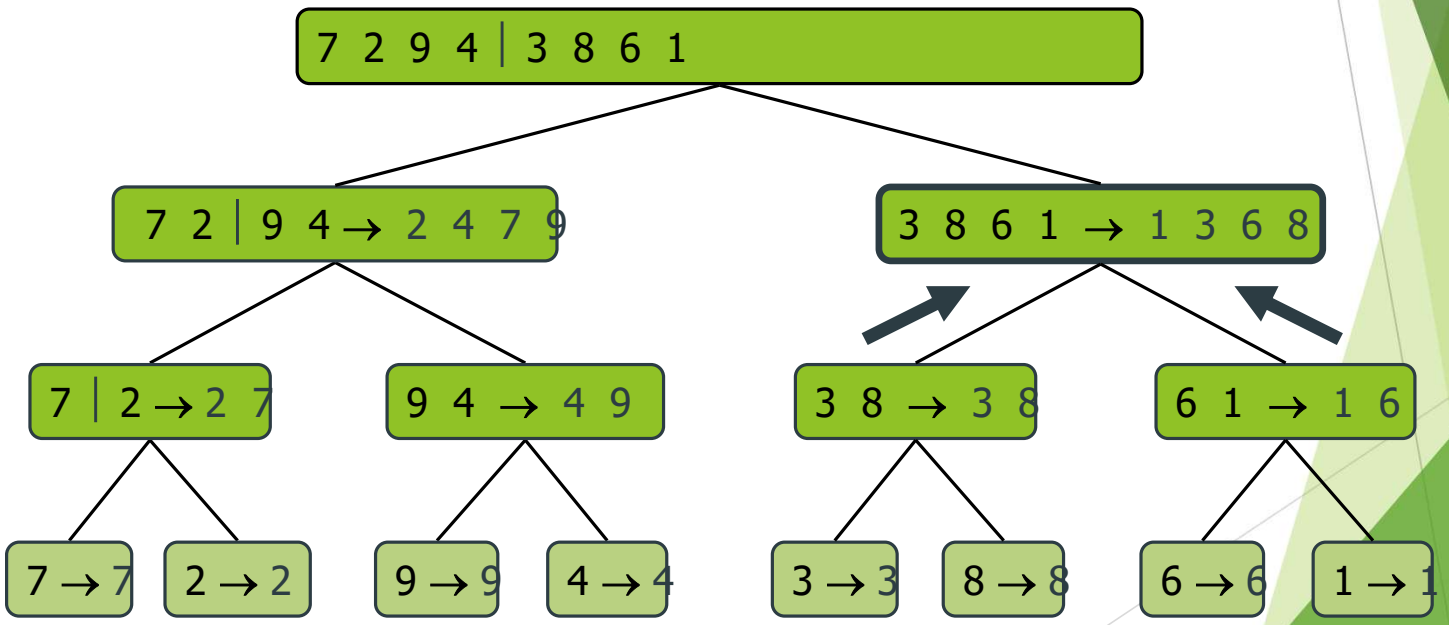
Execution Example (cont.)

- Merge



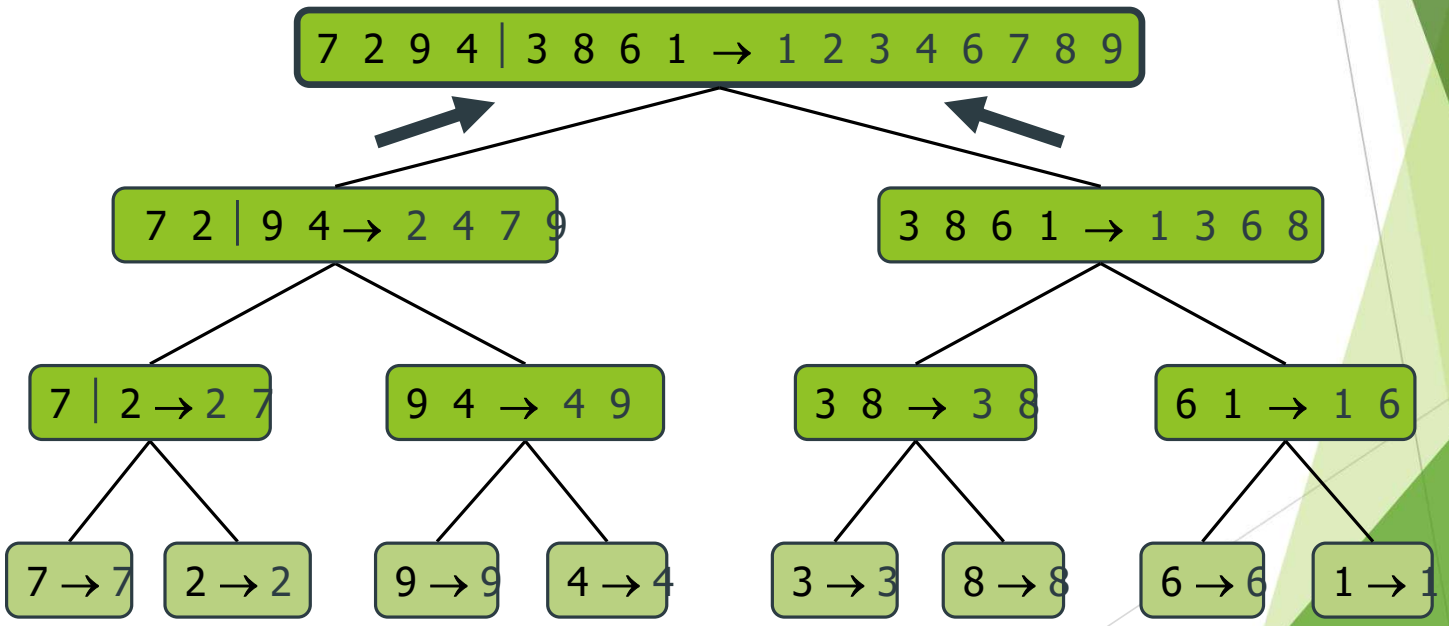
Execution Example (cont.)

- Recursive call, …, merge, merge



Execution Example (cont.)

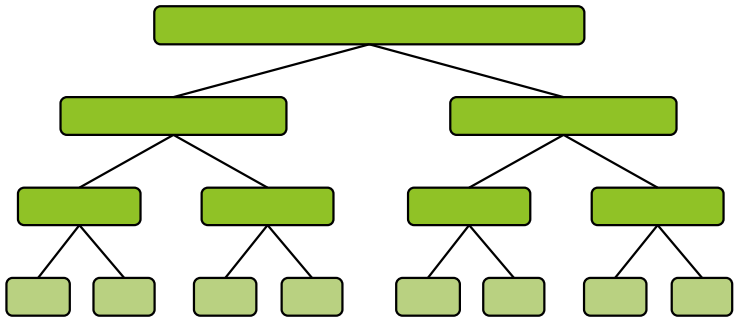
- Merge



Analysis of Merge-Sort

- The height h of the merge-sort tree is $O(\log n)$
  - at each recursive call we divide in half the sequence,
- The overall amount or work done at the nodes of depth i is $O(n)$
  - we partition and merge 2^i sequences of size $n/2^i$
  - we make 2^{i+1} recursive calls
- Thus, the total running time of merge-sort is $O(n \log n)$

depth	#seqs	size
0	1	n
1	2	$n/2$
i	2^i	$n/2^i$
…	…	…



Compute Sum of 1…n

- Problem: Running sum ($\sum_{1}^{n} i$)

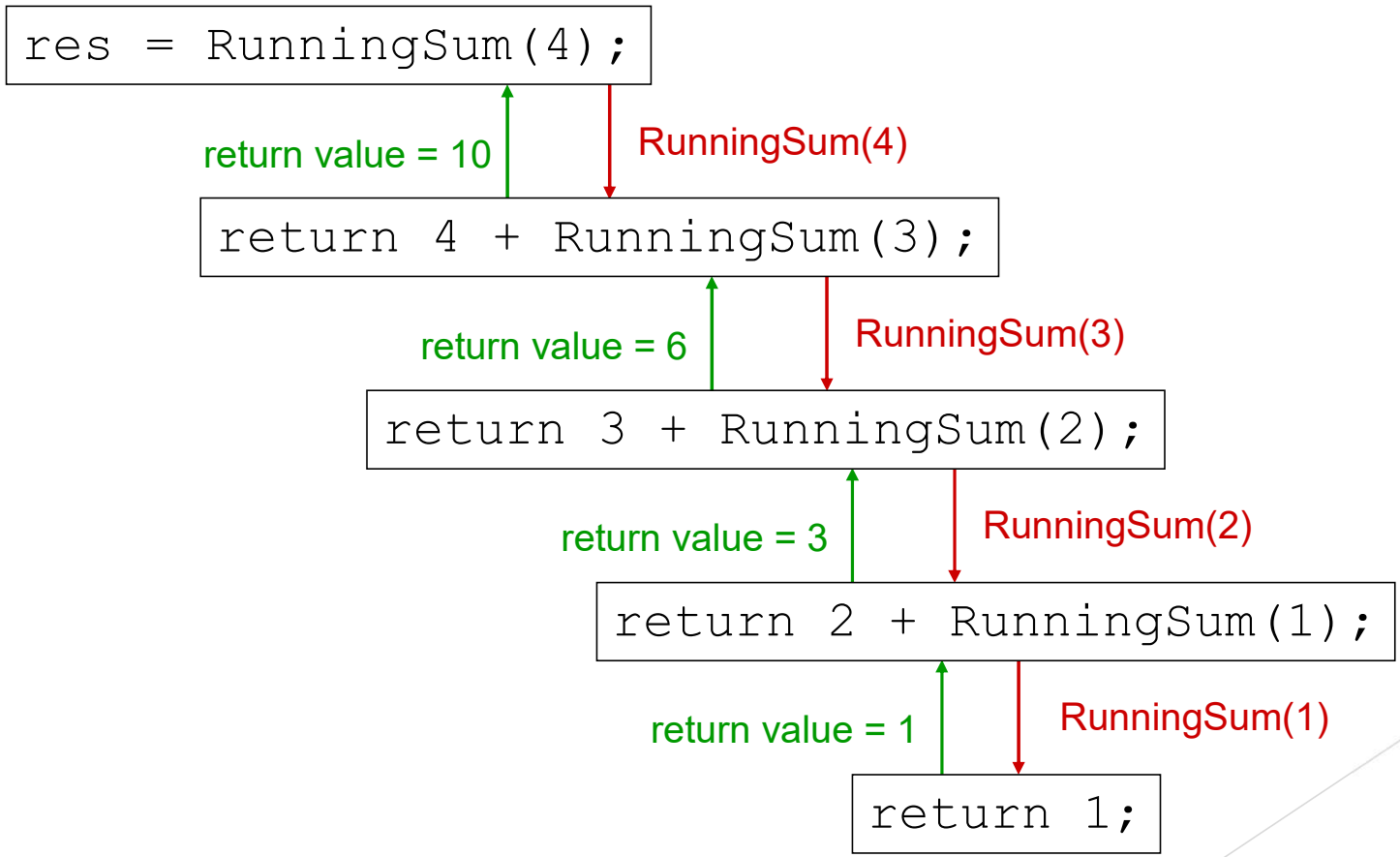
Recursive solution:

RunningSum(1) = 1
RunningSum(n) = n + RunningSum(n-1)

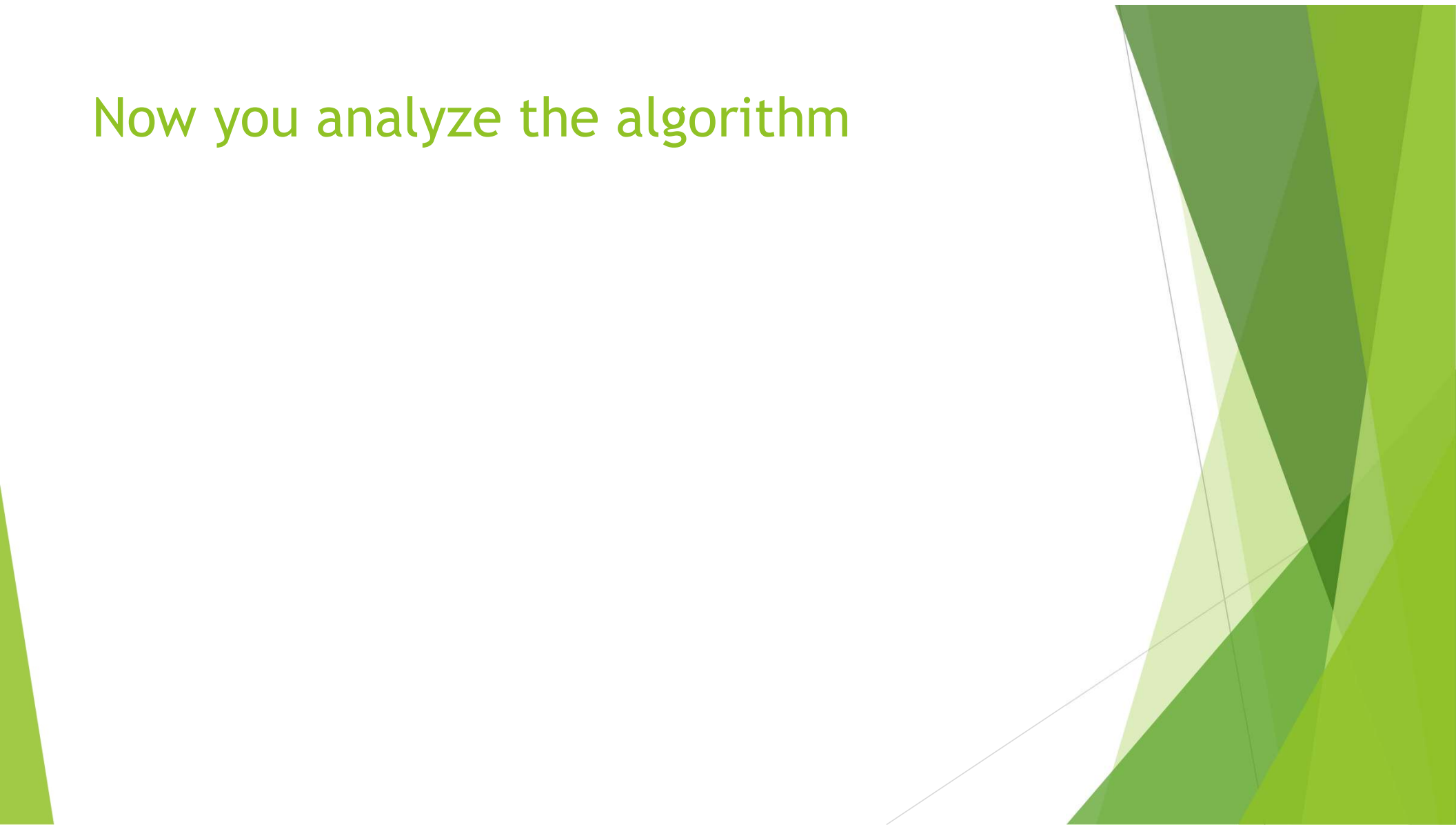
Recursive algorithm:

```
int RunningSum(int n) {
  declare sum
  if (n equal to 1)
    return 1;
  else
    sum = n + RunningSum(n-1)
  return sum
}
```

Executing RunningSum



Now you analyze the algorithm



Compute power

- Recursive definition of b^n :

  $b^n = 1$ if $n = 0$

  $b^n = b \times b^{n-1}$ if $n > 0$

- **Simple recursive power algorithm:**

  To compute b^n:

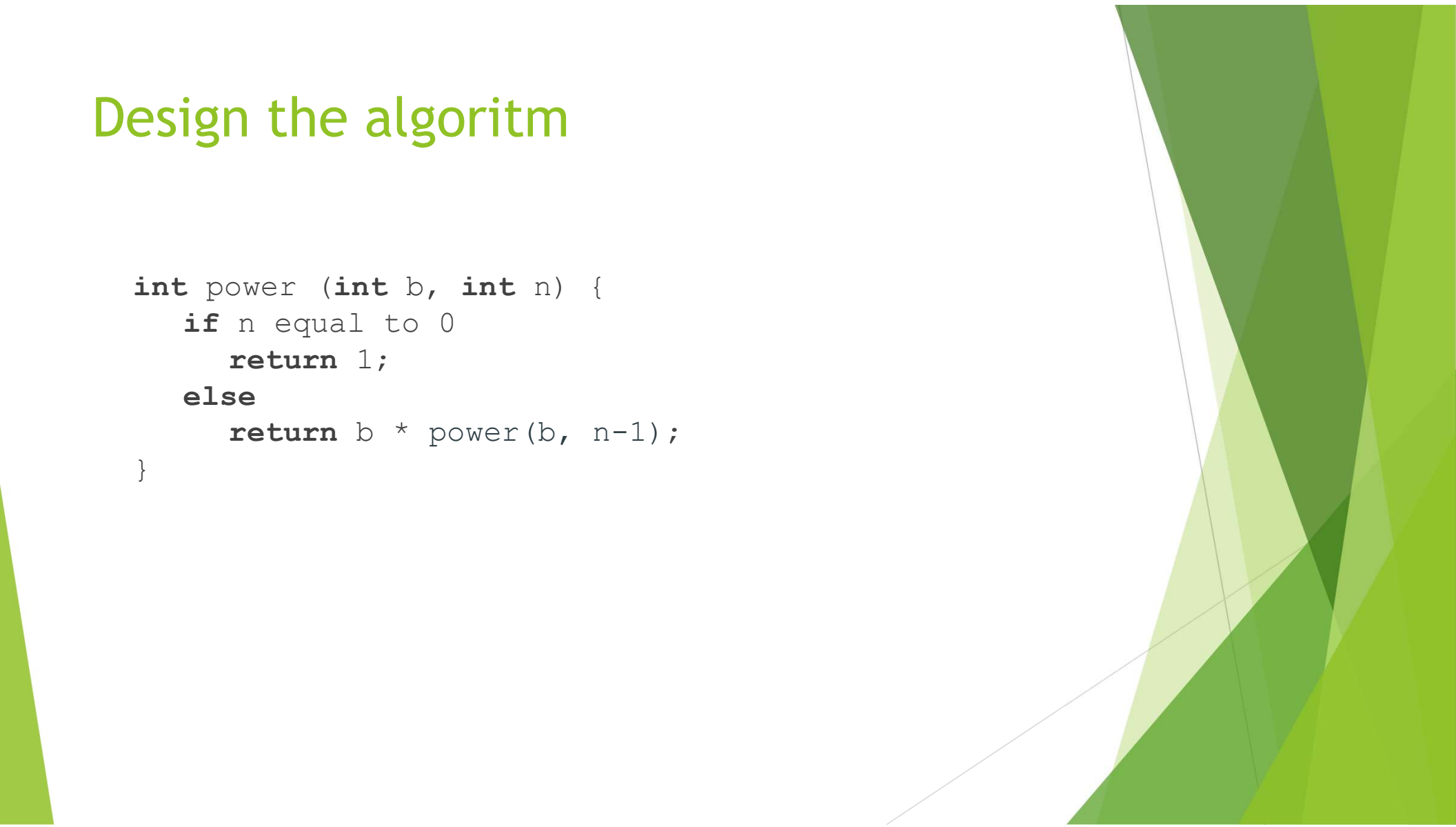
  1. If $n = 0$:
     1.1. Terminate with answer 1.
  2. If $n > 0$:
     2.1. Terminate with answer $b \times b^{n-1}$.

Easy case: solved directly.

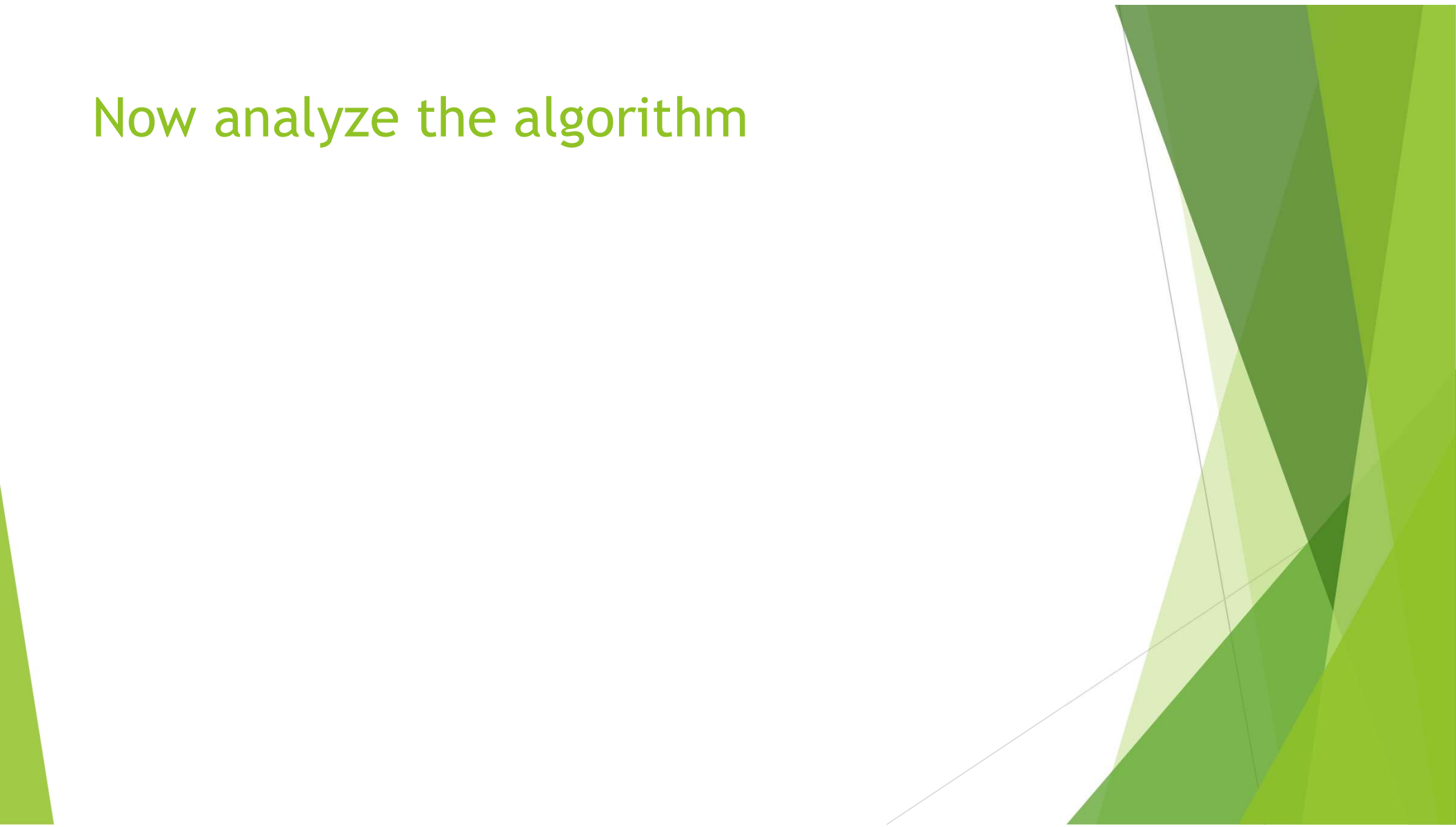
Hard case: solved by computing b^{n-1}, which is easier since $n-1$ is closer than n to 0.

Design the algoritm

```
int power (int b, int n) {
   if n equal to 0
      return 1;
   else
      return b * power(b, n-1);
}
```



Now analyze the algorithm



A smart solution

- Idea: $b^{1000} = b^{500} \times b^{500}$, and $b^{1001} = b \times b^{500} \times b^{500}$.
- Alternative recursive definition of b^n:

$$
\begin{array}{lll}
b^n = 1 & & \text{if } n = 0 \\
b^n = b^{n/2} \times b^{n/2} & & \text{if } n > 0 \text{ and } n \text{ is even} \\
b^n = b \times b^{n/2} \times b^{n/2} & & \text{if } n > 0 \text{ and } n \text{ is odd}
\end{array}
$$

(Recall: $n/2$ discards the remainder if n is odd.)

Recursive solution

- **Smart recursive power algorithm:**

  To compute b^n:

  1. If $n = 0$:
     1.1. Terminate with answer 1.
  2. If $n > 0$:
     2.1. Let p be $b^{n/2}$.
     2.2. If n is even:
          2.2.1. Terminate with answer $p \times p$.
     2.3. If n is odd:
          2.3.1. Terminate with answer $b \times p \times p$.

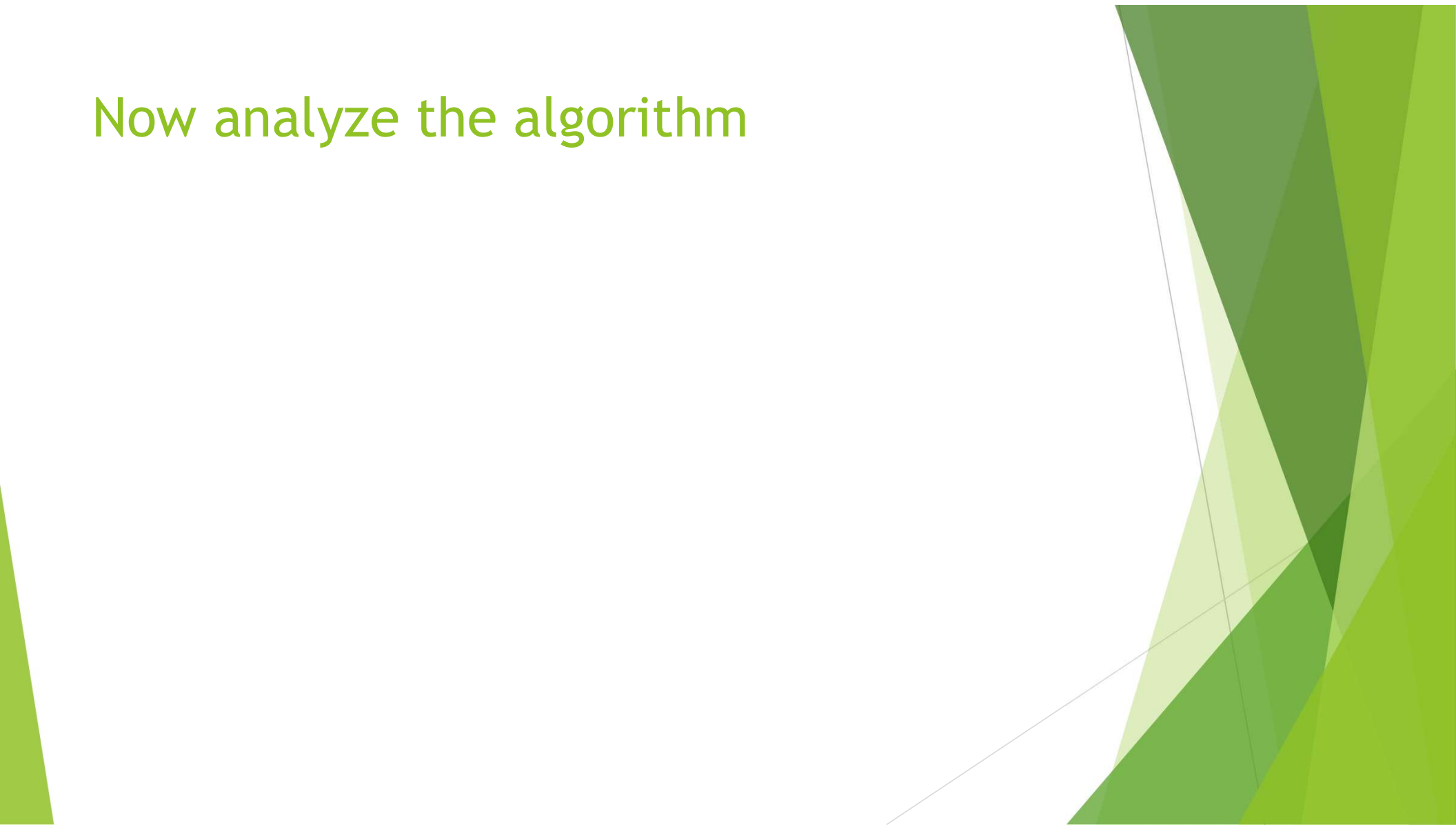
Easy case: solved directly.

Hard case: solved by computing $b^{n/2}$, which is easier since $n/2$ is closer than n to 0.

Design an algorithm

```
int power (int b, int n) {
   if (n equal to 0)
      return 1;
   else {
      Set p to power(b, n/2);
      if (n mod 2 equal to 0)
         return p * p;
      else
         return b * p * p;
   }
}
```

Now analyze the algorithm



Analysis

- Analysis (counting multiplications):

  Each recursive power algorithm performs the same number of multiplications as the corresponding non-recursive algorithm. So their time complexities are the same:

	non-recursive	recursive
Simple power algorithm	$O(n)$	$O(n)$
Smart power algorithm	$O(\log n)$	$O(\log n)$

Recursive Squaring

- We can derive a more efficient linearly recursive algorithm by using repeated squaring:

$$p(x,n) = \begin{cases} 1 & \text{If } n = 0 \\ x \cdot p(x,(n-1)/2)^2 & \text{If } n>0 \text{ is odd} \\ p(x,n/2)^2 & \text{If } n > 0 \text{ is even} \end{cases}$$

- For example,

$2^4 = 2^{(4/2)2} = (2^{4/2})^2 = (2^2)^2 = 4^2 = 16$

$2^5 = 2^{1+(4/2)2} = 2(2^{4/2})^2 = 2(2^2)^2 = 2(4^2) = 32$

$2^6 = 2^{(6/2)2} = (2^{6/2})^2 = (2^3)^2 = 8^2 = 64$

$2^7 = 2^{1+(6/2)2} = 2(2^{6/2})^2 = 2(2^3)^2 = 2(8^2) = 128.$

A Recursive Squaring Method

Algorithm Power(x, n):

Input: A number x and integer $n = 0$

Output: The value x^n

if $n = 0$ **then**

return 1

if n is odd **then**

y = Power(x, (n - 1)/ 2)

return $x \cdot y \cdot y$

else

y = Power(x, n/ 2)

return $y \cdot y$

Analyzing the algorithm

Algorithm Power(x, n):
Input: A number x and integer n = 0
Output: The value x^n
if n = 0 **then**
return 1
if n is odd **then**
y = Power(x, (n - 1)/ 2)
return $x \cdot y \cdot y$
else
y = Power(x, n/ 2)
return $y \cdot y$

Each time we make a recursive call we halve the value of n; hence, we make log n recursive calls. That is, this method runs in O(log n) time.

It is important that we used a variable twice here rather than calling the method twice.

Exercises



Fibonacci sequence problem

The Fibonacci sequence:

{fn } = 0,1,1,2,3,5,8,13,21,34,55

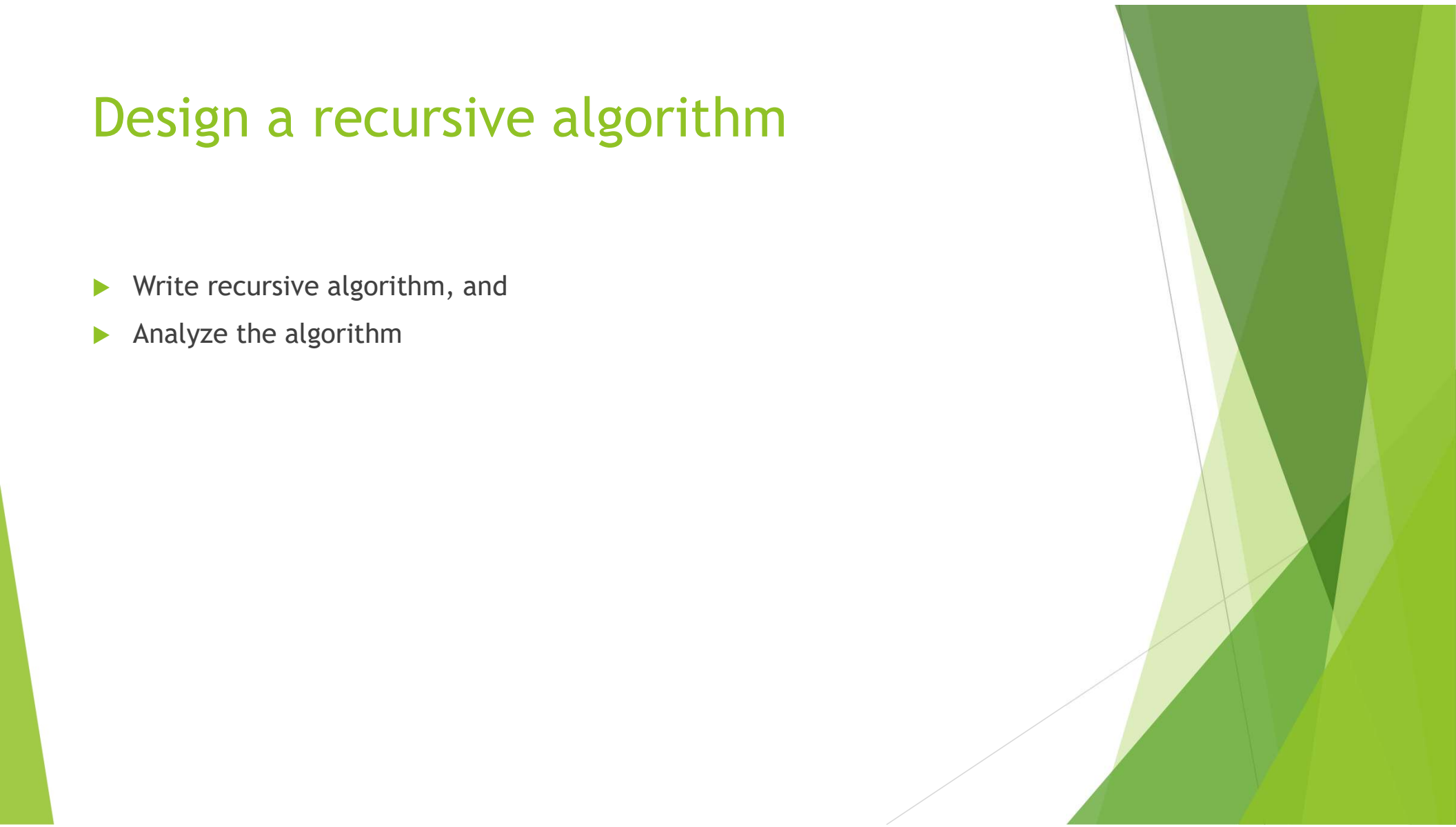
- Mathematical Definition:

$$f(n) = f(n-1) + f(n-2)$$
$$f(1) = 1$$
$$f(0) = 1$$

- In other words, the n-th Fibonacci number is the sum of the previous two Fibonacci numbers.

Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm



Analyze the fib algorithm

- Let us analyze the following recursion

```
long fib(int n){
    if (n <= 1)
        return 1;
    return fib(n-1) + fib(n-2);
}
```

- $T(N) = T(N-1) + T(N-2) + 2$
- By induction, it is possible to show that $fib(N) < (5/3)^N$ and $fib(N) \geq (3/2)^N$
- *So the running time grows exponentially. By using a for loop, the running time can be reduced substantially*

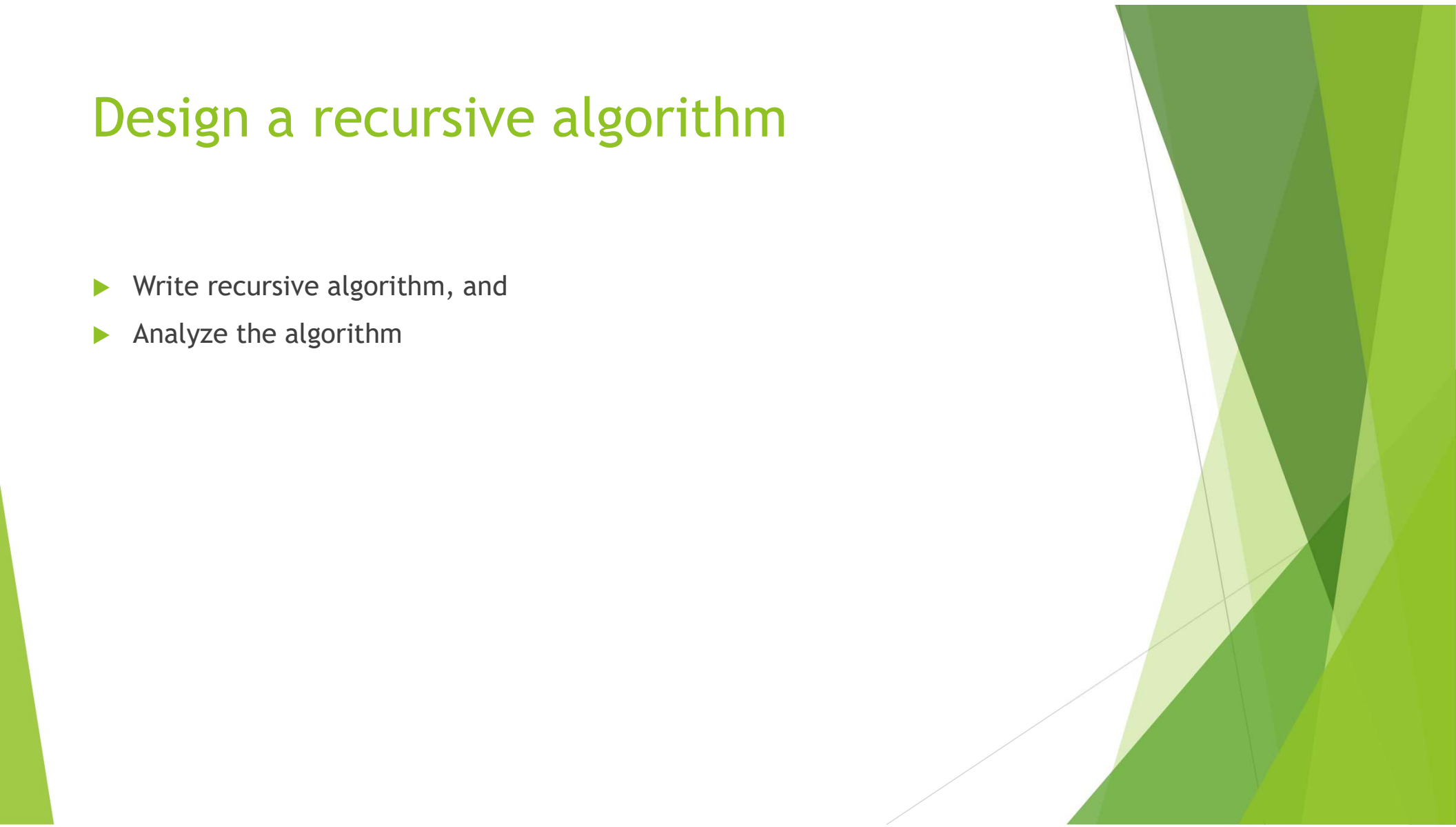
Linear search problem

PROBLEM: Start with a collection of names N_1, N_2, ..., N_{10000}, and corresponding telephone numbers T_1, T_2, ..., T_{10000}. Given a name, Name, find a telephone number for that name if a match on an Ni occurs; otherwise, print "Not Found".

Note: In the book, subscripts are used for N_1, N_2, etc.

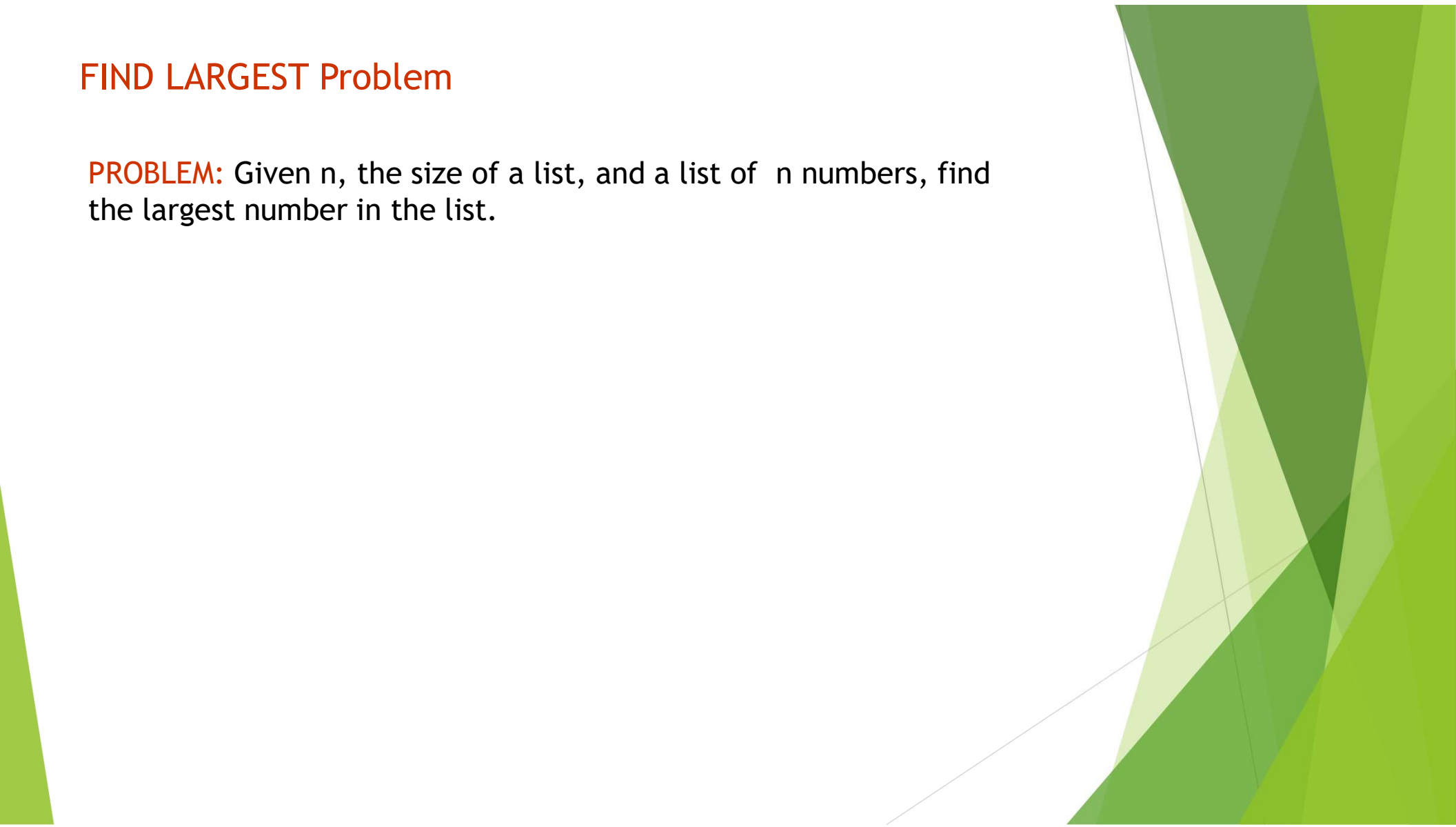
Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm



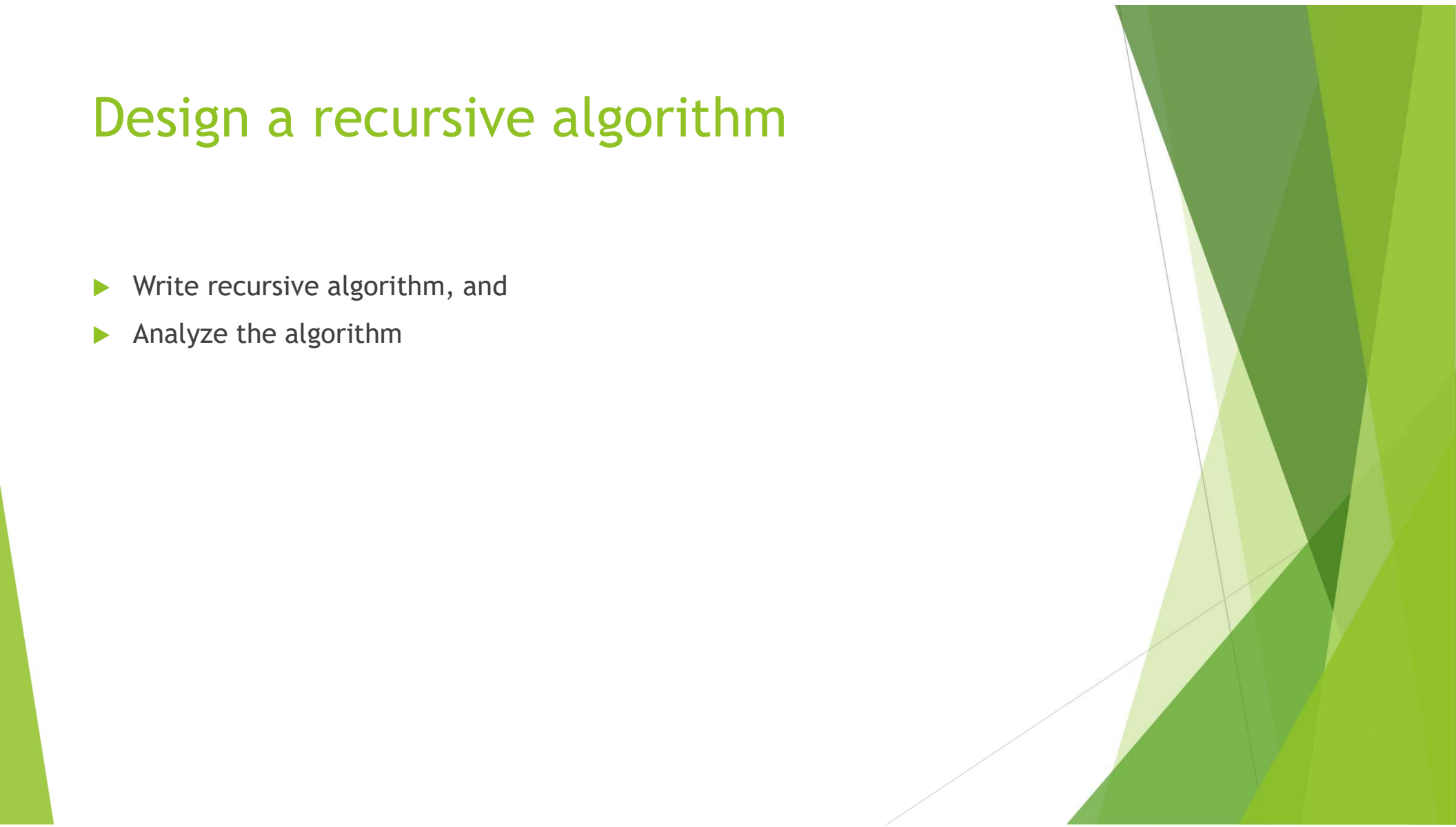
FIND LARGEST Problem

PROBLEM: Given n, the size of a list, and a list of n numbers, find the largest number in the list.



Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm

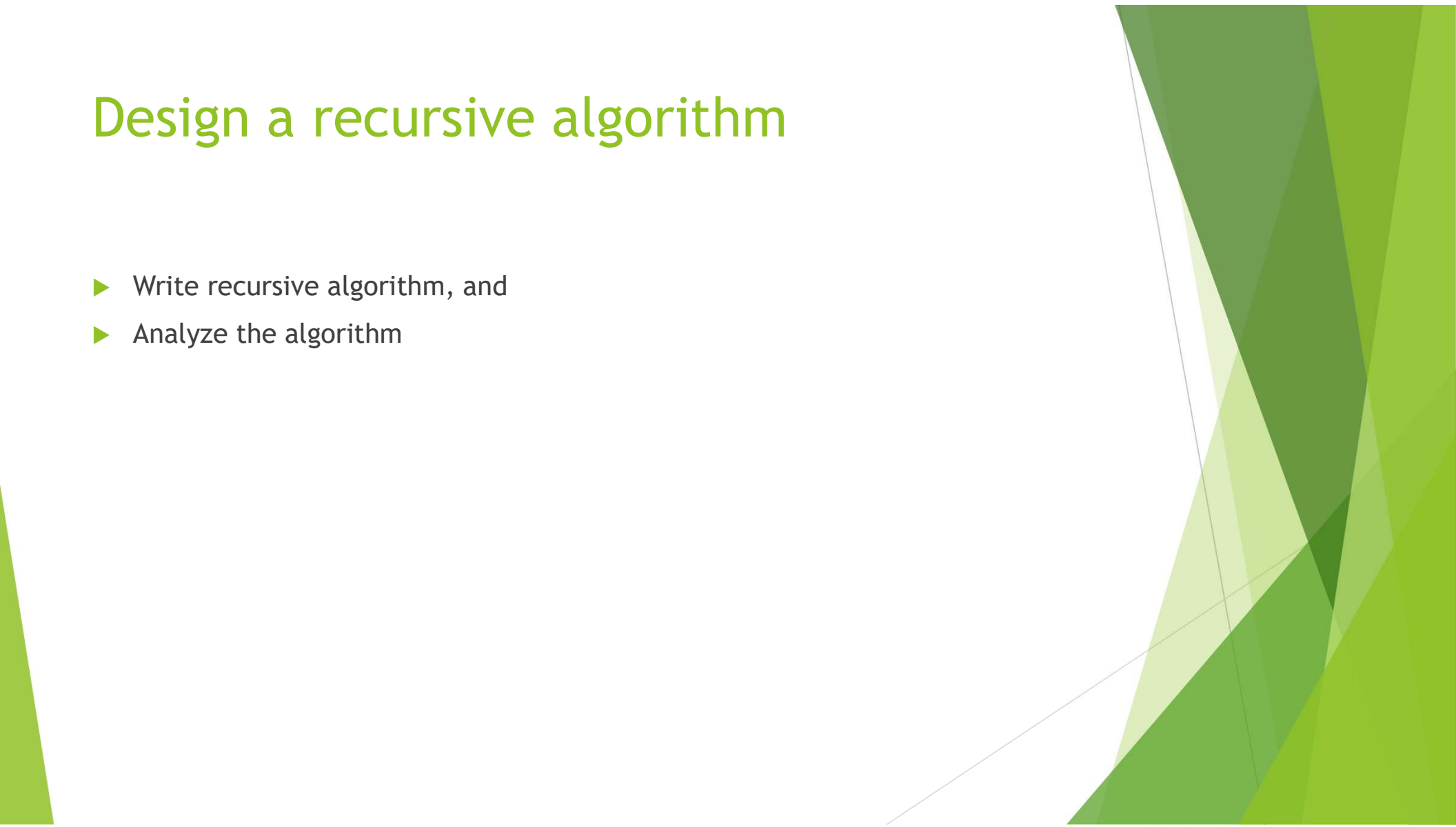


Binary Search problem

- Searching a sorted list
- How can we exploit the structure that a sorted list provides?
  - Start in the middle and determine in which half of the list our target lives
  - Repeat this procedure eliminating half of the remaining list elements on each iteration
  - What is the maximum number of comparisons required here?

Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm



Computing Prefix Averages Problem

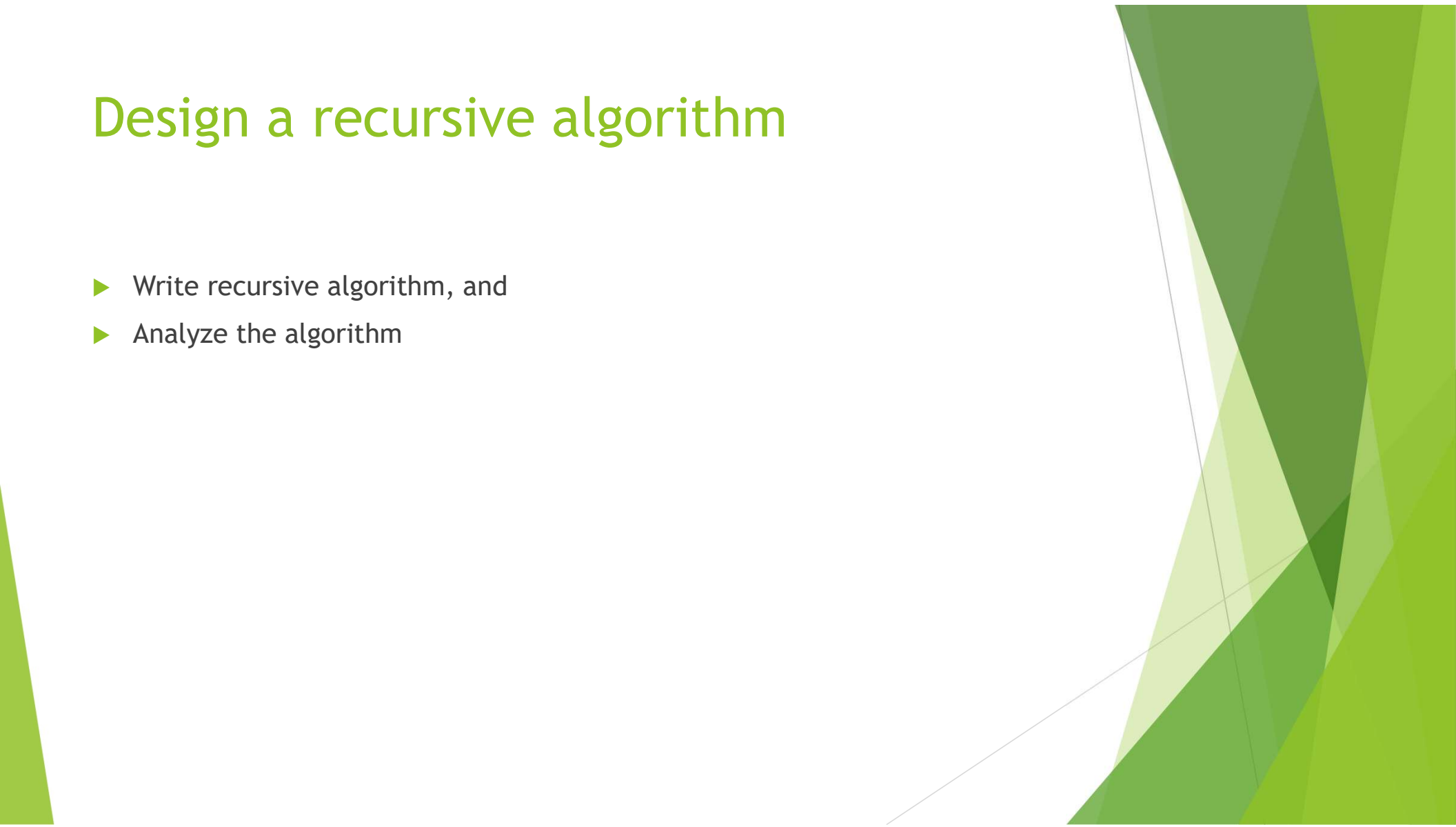
- We further illustrate asymptotic analysis with two algorithms for prefix averages
- The i-th prefix average of an array X is average of the first $(i + 1)$ elements of X:

$$A[i] = (X[0] + X[1] + \ldots + X[i])/(i+1)$$



Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm



Examples of Algorithms – Computing the Greatest Common Divisor of Two Integers

The greatest common divisor of two non-negative, not-both-zero integers m and n, denoted **gcd(m, n)**, is defined as the largest integer that divides both m and n evenly, until the remainder is zero

- Euclid's algorithm is based on repeated application of equality until the second number becomes 0

  gcd(m, n) = gcd(n, m mod n)

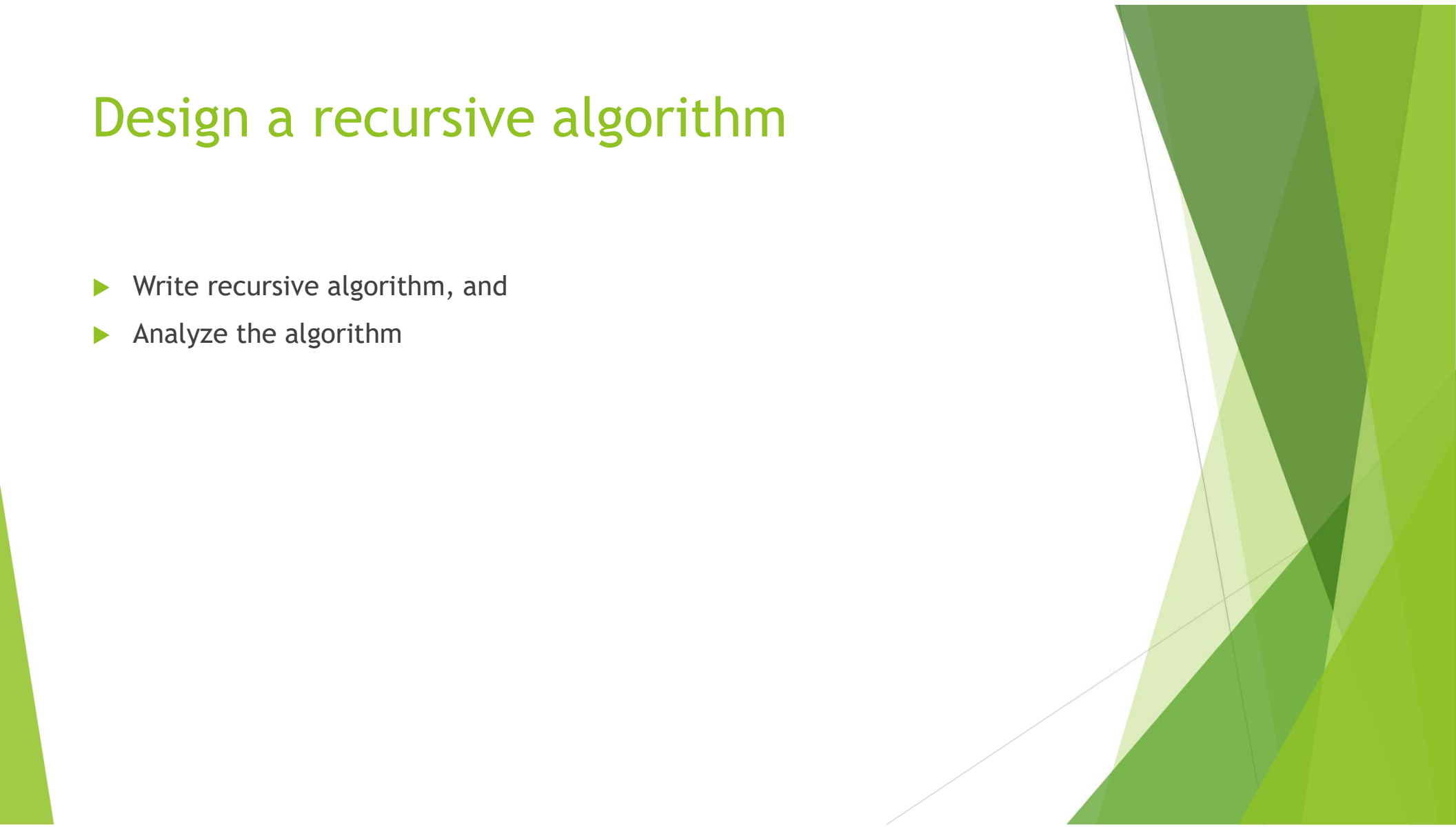
- For example **gcd(60, 24)** can be computed as follows:

  gcd(60, 24) = gcd(24, 12) = gcd(12, 0) = 12

$$\gcd(x, y) = \begin{cases} x, & \text{if } y = 0 \\ \gcd(y, x \bmod y), & \text{otherwise} \end{cases}$$

Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm



Selection Sort

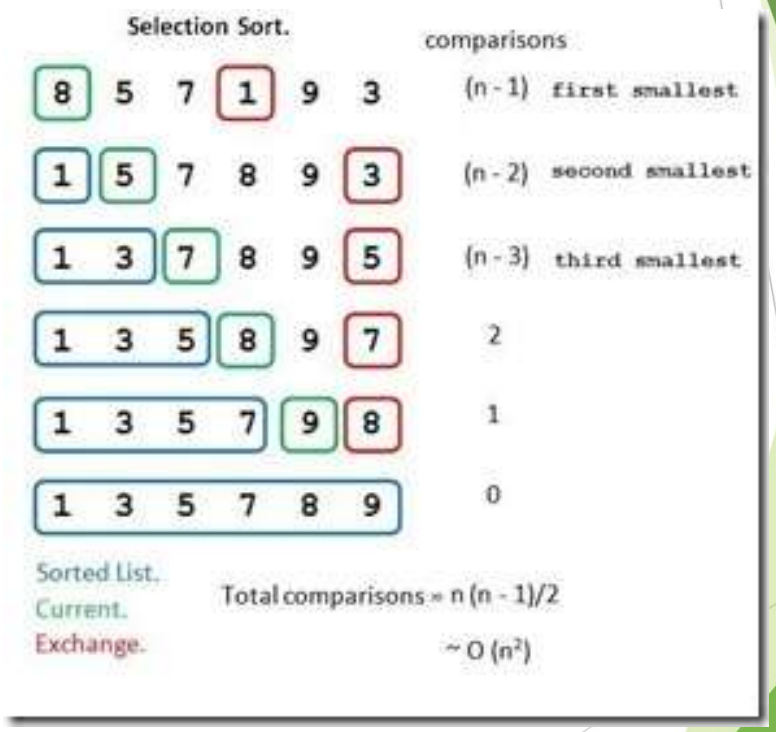
n is the size of unsorted list.

if n is 1,

- array is sorted

else

- (1) find the smallest element in the unsorted array list
- (2) swap the 1st element in the unsorted array with the smallest element
- (3) sort the remaining unsorted array



Design a recursive algorithm

- Write recursive algorithm, and
- Analyze the algorithm

